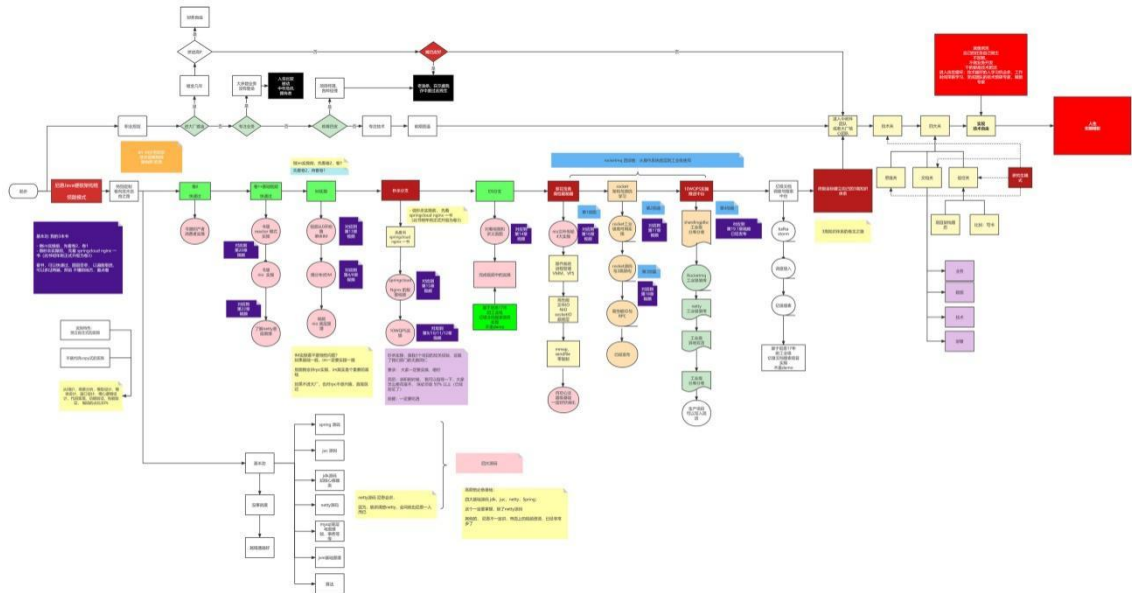


牛逼的职业发展之路

40 岁老架构尼恩用一张图揭秘：Java 工程师的高端职业发展路径，走向食物链顶端的之路

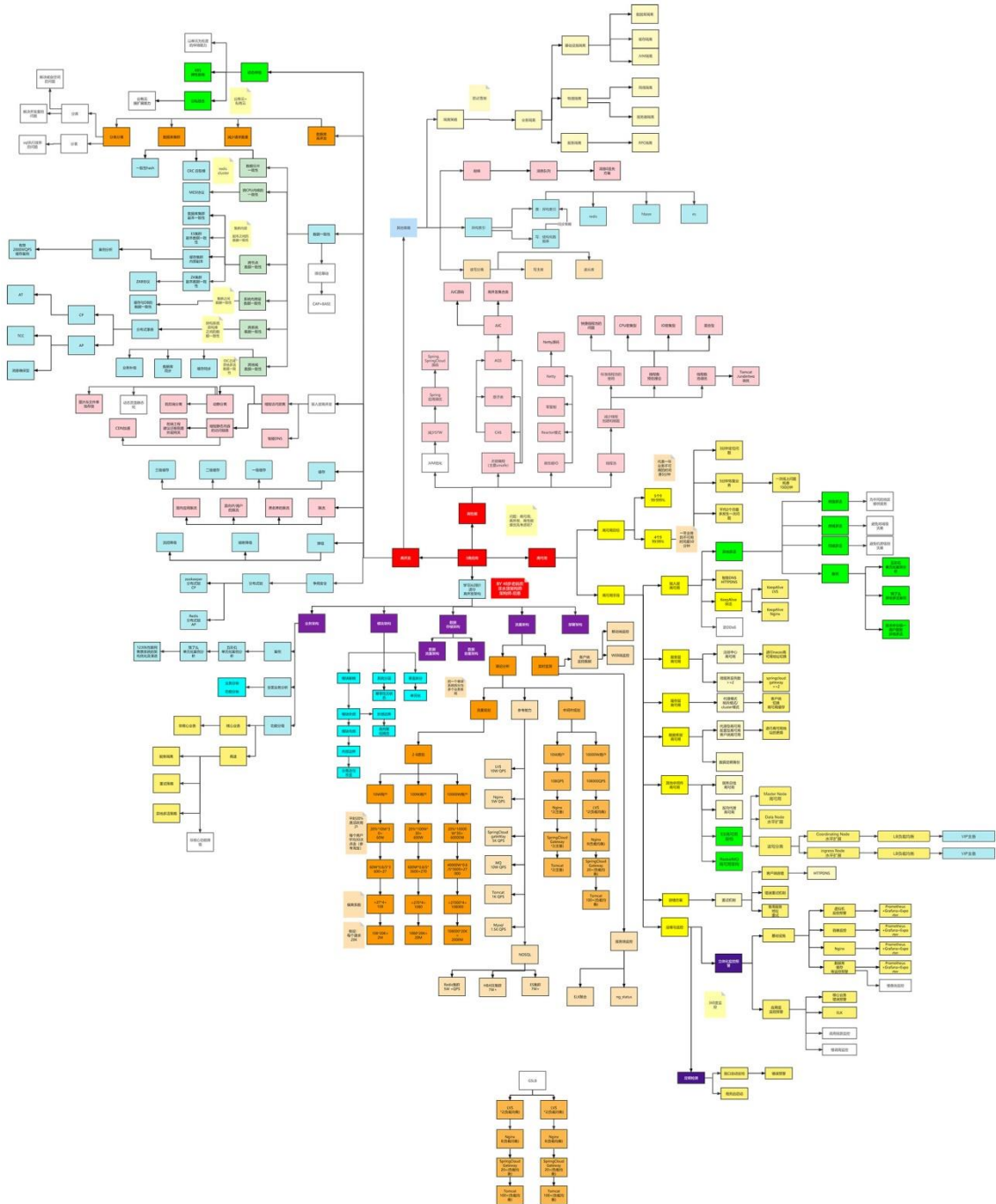
链接：<https://www.processon.com/view/link/618a2b62e0b34d73f7eb3cd7>



史上最全：价值10W的架构师知识图谱

此图梳理于尼恩的多个 3 高生产项目：多个亿级人民币的大型 SAAS 平台和智慧城市项目

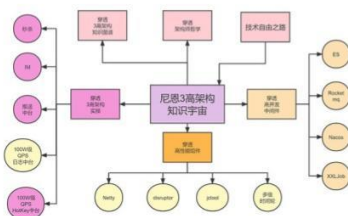
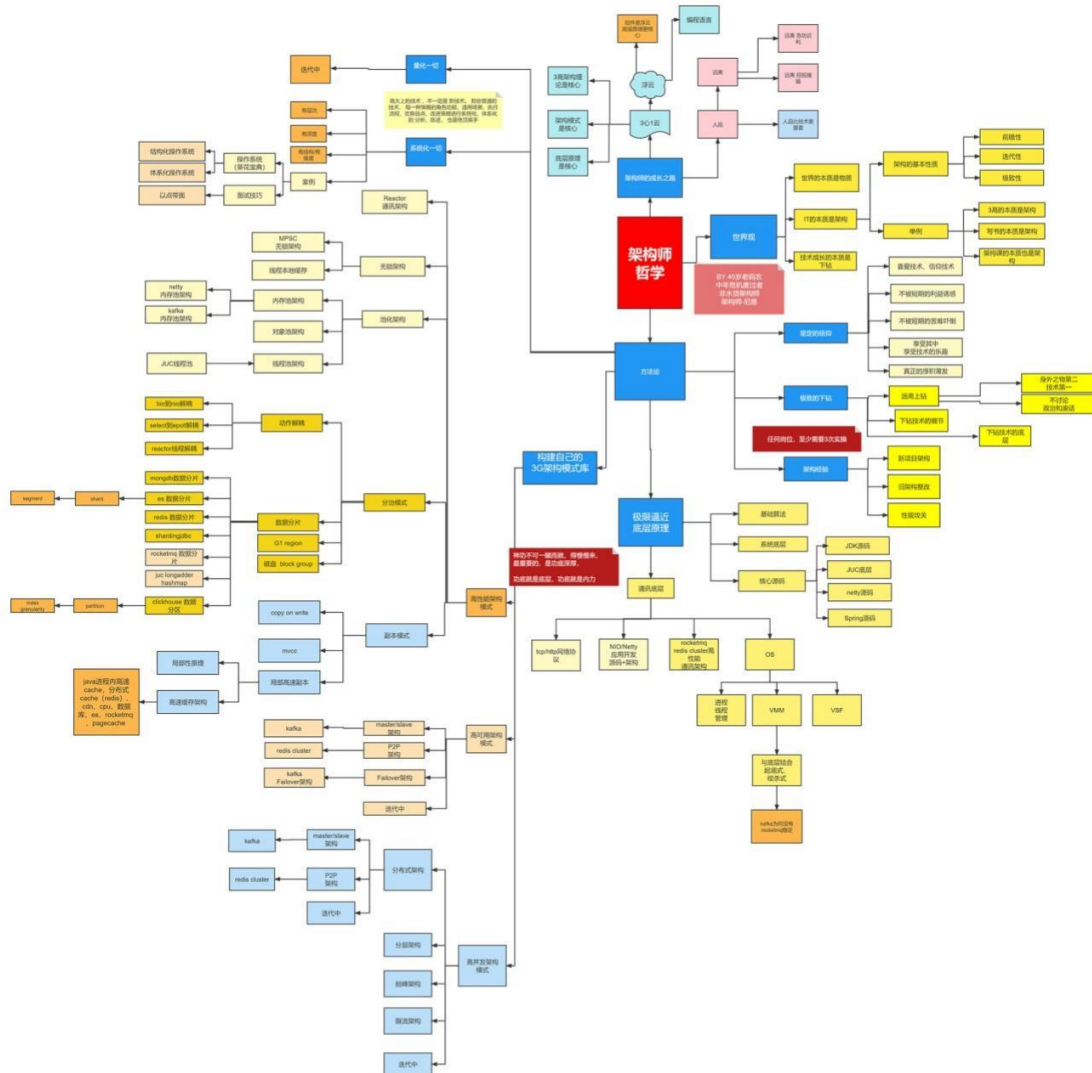
链接：<https://www.processon.com/view/link/60fb9421637689719d246739>



牛逼的架构师哲学

40 岁老架构师尼恩对自己的 20 年的开发、架构经验总结

链接: <https://www.processon.com/view/link/616f801963768961e9d9aec8>



牛逼的3高架构知识宇宙

尼恩 3 高架构知识宇宙，帮助大家穿透 3 高架构，走向技术自由，远离中年危机

链接: <https://www.processon.com/view/link/635097d2e0b34d40be778ab4>



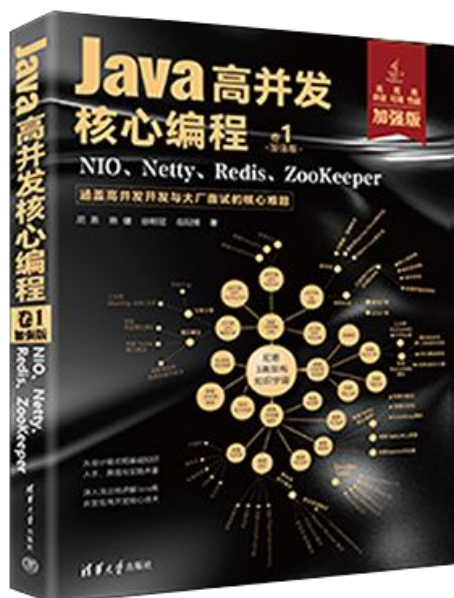
尼恩Java高并发三部曲（卷1加强版）

老版本：《Java 高并发核心编程 卷1：NIO、Netty、Redis、ZooKeeper》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷1 **加强版**：NIO、Netty、Redis、ZooKeeper》

- 由浅入深地剖析了高并发 IO 的底层原理。
- 图文并茂地介绍了 TCP、HTTP、WebSocket 协议的核心原理。
- 细致深入地揭秘了 Reactor 高性能模式。
- 全面介绍了 Netty 框架，并完成单体 IM、分布式 IM 的实战设计。
- 详尽地介绍了 ZooKeeper、Redis 的使用，以帮助提升高并发、可扩展能力

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



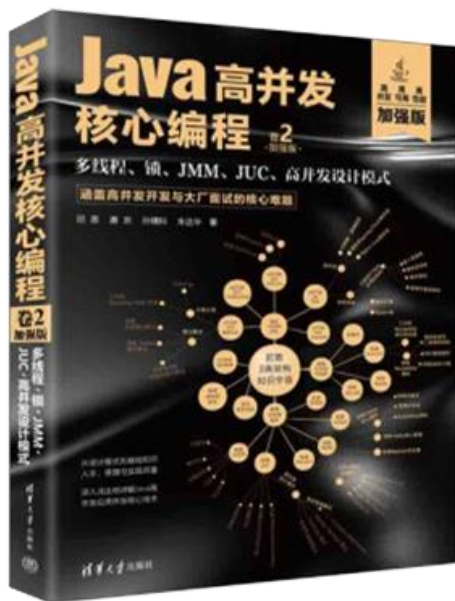
尼恩Java高并发三部曲（卷2加强版）

老版本：《Java 高并发核心编程 卷2：多线程、锁、JMM、JUC、高并发设计模式》
（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷2 **加强版**：多线程、锁、JMM、JUC、高并发设计模式》

- 由浅入深地剖析了 Java 多线程、线程池的底层原理。
- 总结了 IO 密集型、CPU 密集型线程池的线程数预估算法。
- 图文并茂地介绍了 Java 内置锁、JUC 显式锁的核心原理。
- 细致深入地揭秘了 JMM 内存模型。
- 全面介绍了 JUC 框架的设计模式与核心原理，并完成其高核心组件的实战介绍。
- 详尽地介绍了高并发设计模式的使用，以帮助提升高并发、可扩展能力

详情参阅：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



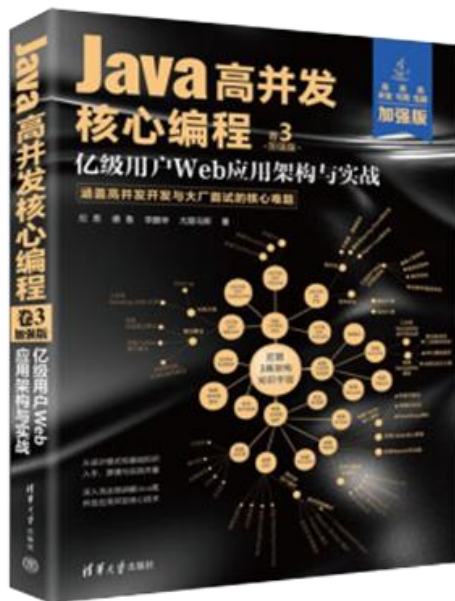
尼恩Java高并发三部曲（卷3加强版）

老版本：《SpringCloud Nginx 高并发核心编程》（已经过时，不建议购买）

新版本：《Java 高并发核心编程 卷3 **加强版**：亿级用户 Web 应用架构与实战》

- 在当今的面试场景中，3 高知识是大家面试必备的核心知识，本书基于亿级用户 3 高 Web 应用的架构分析理论，为大家对 3 高架构系统做一个系统化和清晰化的介绍。
- 从 Java 静态代理、动态代理模式入手，抽丝剥茧地解读了 Spring Cloud 全家桶中 RPC 核心原理和执行过程，这是高级 Java 工程师面试必备的基础知识。
- 从Reactor 反应器模式入手，抽丝剥茧地解读了Nginx 核心思想和各配置项的底层知识和原理，这是高级 Java 工程师、架构师面试必备的基础知识。
- 从观察者模式入手，抽丝剥茧地解读了 RxJava、Hystrix 的核心思想和使用方法，这也是高级 Java 工程师、架构师面试必备的基础知识。

详情：<https://www.cnblogs.com/crazymakercircle/p/16868827.html>



专题34：elasticsearch面试题（史上最全、定期更新）

本文版本说明：V3.0

此文的格式，由markdown 通过程序转成而来，由于很多表格，没有来的及调整，出现一个格式问题，尼恩在此给大家道歉啦。

由于社群很多小伙伴，在面试，不断的交流最新的面试难题，所以，《Java面试红宝书》，后面会不断升级，迭代。

本专题，作为 《Java面试红宝书》专题之一，《Java面试红宝书》一共**30个面试专题**，后续还会增加

《尼恩面试宝典》升级的规划为：

后续基本上，**每一个月，都会发布一次**，最新版本，可以扫描扫架构师尼恩微信，发送“领取电子书”获取。

尼恩的微信二维码在哪里呢？请参见文末

面试问题交流说明：

如果遇到面试难题，或者职业发展问题，或者中年危机问题，都可以来 疯狂创客圈社群交流，

入交流群，加尼恩微信即可，发送“**入群**”，

尼恩微信请打开语雀扫码 <https://www.yuque.com/crazymakercircle/gkkw8s/khigna>



ElasticSearch基础：

一、Elasticsearch 的基本概念：

1、什么是Elasticsearch：

Elasticsearch 是基于 Lucene 的 Restful 的分布式实时全文搜索引擎，每个字段都被索引并可被搜索，可以快速存储、搜索、分析海量的数据。

全文检索是指对每一个词建立一个索引，指明该词在文章中出现的时间和位置。当查询时，根据事先建立的索引进行查找，并将查找的结果反馈给用户的检索方式。这个过程类似于通过字典中的检索字表查字的过程。

2、Elasticsearch 的基本概念：

(1) index 索引：索引类似于mysql 中的数据库，Elasticsearch 中的索引是存在数据的地方，包含了一堆有相似结构的文档数据。

(2) type 类型：类型是用来定义数据结构，可以认为是 mysql 中的一张表，type 是 index 中的一个逻辑数据分类

(3) document 文档：类似于 MySQL 中的一行，不同之处在于 ES 中的每个文档可以有不同的字段，但是对于通用字段应该具有相同的数据类型，文档是 es 中的最小数据单元，可以认为一个文档就是一条记录。

(4) Field 字段：Field 是 Elasticsearch 的最小单位，一个 document 里面有多多个 field

(5) shard 分片：单台机器无法存储大量数据，es 可以将一个索引中的数据切分为多个 shard，分布在多台服务器上存储。有了 shard 就可以横向扩展，存储更多数据，让搜索和分析等操作分布到多台服务器上去执行，提升吞吐量和性能。

(6) replica 副本：任何服务器随时可能故障或宕机，此时 shard 可能会丢失，通过创建 replica 副本，可以在 shard 故障时提供备用服务，保证数据不丢失，另外 replica 还可以提升搜索操作的吞吐量。

shard 分片数量在建立索引时设置，设置后不能修改，默认 5 个；replica 副本数量默认 1 个，可随时修改数量；

3、什么是倒排索引：

在搜索引擎中，每个文档都有对应的文档 ID，文档内容可以表示为一系列关键词的集合，例如，某个文档经过分词，提取了 20 个关键词，而通过倒排索引，可以记录每个关键词在文档中出现的次数和出现位置。也就是说，倒排索引是 关键词到文档 ID 的映射，每个关键词都对应着一系列的文件，这些文件中都出现了该关键词。

要注意倒排索引的两个细节：

- 倒排索引中的所有词项对应一个或多个文档
- 倒排索引中的词项 根据字典顺序升序排列

4、doc_values 的作用：

倒排索引虽然可以提高搜索性能，但也存在缺陷，比如我们需要对数据做排序或聚合等操作时，[lucene](#) 会提取所有出现在文档集合的排序字段，然后构建一个排好序的文档集合，而这个步骤是基于内存的，如果排序数据量巨大的话，容易造成内存溢出和性能缓慢。

doc_values 就是 es 在构建倒排索引的同时，会对开启 doc_values 的字段构建一个有序的“document 文档 ==> field value”的列式存储映射，可以看作是以文档维度，实现了根据指定字段进行排序和聚合的功能，降低对内存的依赖。另外 doc_values 保存在操作系统的磁盘中，当 doc_values 大于节点的可用内存，ES 可以从操作系统页缓存中加载或弹出，从而避免发生内存溢出的异常，但如果 docValues 远小于节点的可用内存，操作系统就自然将所有 doc_values 存于内存中（堆外内存），有助于快速访问。

5、text 和 keyword 类型的区别：

两个类型的区别主要是分词：keyword 类型是不会分词的，直接根据字符串内容建立倒排索引，所以 keyword 类型的字段只能通过精确值搜索到；Text 类型在存入 Elasticsearch 的时候，会先分词，然后根据分词后的内容建立倒排索引

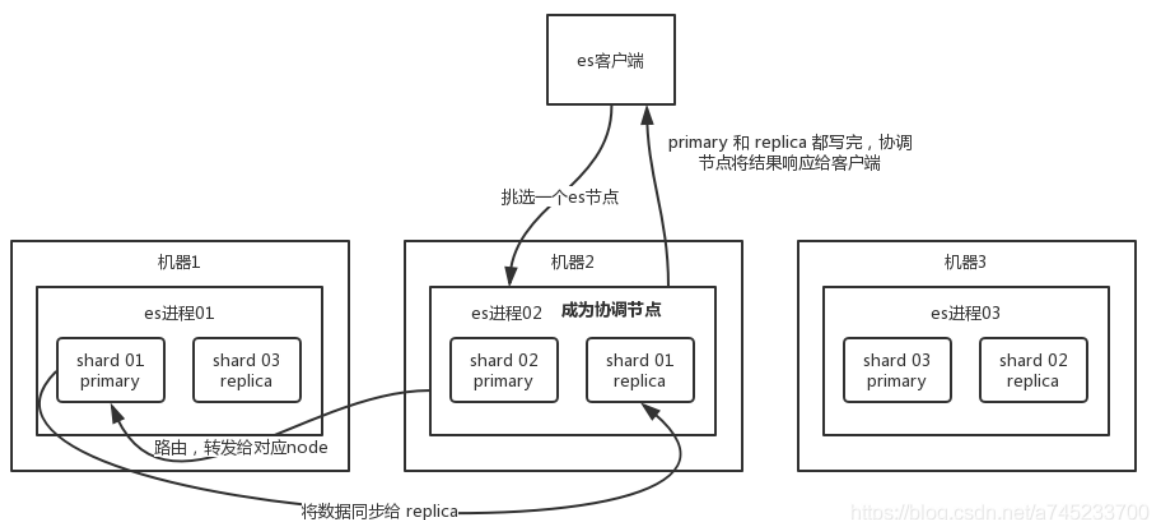
6、query 和 filter 的区别？

(1) query：查询操作不仅仅会进行查询，还会计算分值，用于确定相关度；

(2) filter：查询操作仅判断是否满足查询条件，不会计算任何分值，也不会关心返回的排序问题，同时，filter 查询的结果可以被缓存，提高性能。

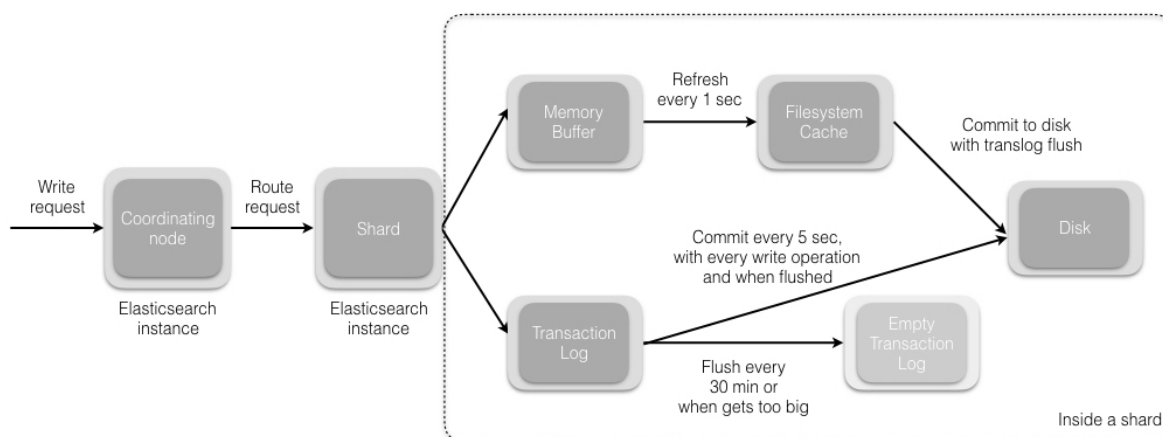
二、ES 的写入流程：

1、ES写数据的整体流程：



- (1) 客户端选择 ES 的某个 node 发送请求过去, 这个 node 就是协调节点 coordinating node
- (2) coordinating node 对 document 进行路由, 将请求转发给对应的 node (有 primary shard)
- (3) 实际的 node 上的 primary shard 处理请求, 然后将数据同步到 replica node
- (4) coordinating node 等到 primary node 和所有 replica node 都执行成功之后, 最后返回响应结果给客户端。

2、ES主分片写数据的详细流程：



(1) 主分片先将数据写入ES的 memory buffer, 然后定时 (默认1s) 将 memory buffer 中的数据写入一个新的 segment 文件中, 并进入操作系统缓存 Filesystem cache (同时清空 memory buffer), 这个过程就叫做 refresh; 每个 segment 文件实际上是一些倒排索引的集合, 只有经历了 refresh 操作之后, 这些数据才能变成可检索的。

ES 的近实时性: 数据存在 memory buffer 时是搜索不到的, 只有数据被 refresh 到 Filesystem cache 之后才能被搜索到, 而 refresh 是每秒一次, 所以称 es 是近实时的; 可以手动调用 es 的 api 触发一次 refresh 操作, 让数据马上可以被搜索到;

(2) 由于 memory Buffer 和 Filesystem Cache 都是基于内存, 假设服务器宕机, 那么数据就会丢失, 所以 ES 通过 translog 日志文件来保证数据的可靠性, 在数据写入 memory buffer 的同时, 将数据也写入 translog 日志文件中, 当机器宕机重启时, es 会自动读取 translog 日志文件中的数据, 恢复到 memory buffer 和 Filesystem cache 中去。

ES 数据丢失的问题：translog 也是先写入 Filesystem cache，然后默认每隔 5 秒刷一次到磁盘中，所以默认情况下，可能有 5 秒的数据会仅仅停留在 memory buffer 或者 translog 文件的 Filesystem cache 中，而不在磁盘上，如果此时机器宕机，会丢失 5 秒钟的数据。也可以将 translog 设置成每次写操作必须是直接 fsync 到磁盘，但是性能会差很多。

(3) flush 操作：不断重复上面的步骤，translog 会越来越大，不过 translog 文件默认每30分钟或者 阈值超过 512M 时，就会触发 commit 操作，即 flush操作，将 memory buffer 中所有的数据写入新的 segment 文件中，并将内存中所有的 segment 文件全部落盘，最后清空 translog 事务日志。

- ① 将 memory buffer 中的数据 refresh 到 Filesystem Cache 中去，清空 buffer；
- ② 创建一个新的 commit point（提交点），同时强行将 Filesystem Cache 中目前所有的数据都 fsync 到磁盘文件中；
- ③ 删除旧的 translog 日志文件并创建一个新的 translog 日志文件，此时 commit 操作完成

更多 ES 的数据写入流程的说明欢迎阅读这篇文章：[ElasticSearch搜索引擎：数据的写入流程](#)

三、ES的更新和删除流程：

删除和更新都是写操作，但是由于 Elasticsearch 中的文档是不可变的，因此不能被删除或者改动以展示其变更；所以 ES 利用 .del 文件 标记文档是否被删除，磁盘上的每个段都有一个相应的.del 文件

(1) 如果是删除操作，文档其实并没有真的被删除，而是在 .del 文件中被标记为 deleted 状态。该文档依然能匹配查询，但是会在结果中被过滤掉。

(2) 如果是更新操作，就是将旧的 doc 标识为 deleted 状态，然后创建一个新的 doc。

memory buffer 每 refresh 一次，就会产生一个 segment 文件，所以默认情况下是 1s 生成一个 segment 文件，这样下来 segment 文件会越来越多，此时会定期执行 merge。每次 merge 的时候，会将多个 segment 文件合并成一个，同时这里会将标识为 deleted 的 doc 给物理删除掉，不写入到新的 segment 中，然后将新的 segment 文件写入磁盘，这里会写一个 commit point，标识所有新的 segment 文件，然后打开 segment 文件供搜索使用，同时删除旧的 segment 文件

有关segment段合并过程，欢迎阅读这篇文章：[Elasticsearch搜索引擎：ES的segment段合并原理](#)

四、ES的搜索流程：

搜索被执行成一个两阶段过程，即 Query Then Fetch：

1、Query阶段：

客户端发送请求到 coordinate node，协调节点将搜索请求广播到所有的 primary shard 或 replica，每个分片在本地执行搜索并构建一个匹配文档的大小为 from + size 的优先队列。接着每个分片返回各自优先队列中 所有 docId 和 打分值 给协调节点，由协调节点进行数据的合并、排序、分页等操作，产出最终结果。

2、Fetch阶段：

协调节点根据 Query阶段产生的结果，去各个节点上查询 docId 实际的 document 内容，最后由协调节点返回结果给客户端。

- coordinate node 对 doc id 进行哈希路由，将请求转发到对应的 node，此时会使用 round-robin 随机轮询算法，在 primary shard 以及其所有 replica 中随机选择一个，让读请求负载均衡。

- 接收请求的 node 返回 document 给 coordinate node 。
- coordinate node 返回 document 给客户端。

Query Then Fetch 的搜索类型在文档相关性打分的时候参考的是本分片的数据，这样在文档数量较少的时候可能不够准确，DFS Query Then Fetch 增加了一个预查询的处理，询问 Term 和 Document frequency，这个评分更准确，但是性能会变差。

五、ES在高并发下如何保证读写一致性？

(1) 对于更新操作：可以通过版本号使用乐观并发控制，以确保新版本不会被旧版本覆盖

每个文档都有一个_version 版本号，这个版本号在文档被改变时加一。Elasticsearch使用这个_version 保证所有修改都被正确排序，当一个旧版本出现在新版本之后，它会被简单的忽略。

利用_version的这一优点确保数据不会因为修改冲突而丢失，比如指定文档的version来做更改，如果那个版本号不是现在的，我们的请求就失败了。

(2) 对于写操作，一致性级别支持 quorum/one/all，默认为 quorum，即只有当大多数分片可用时才允许写操作。但即使大多数可用，也可能存在因为网络等原因导致写入副本失败，这样该副本被认为故障，副本将会在一个不同的节点上重建。

- one：写操作只要有一个primary shard是active活跃可用的，就可以执行
- all：写操作必须所有的primary shard和replica shard都是活跃可用的，才可以执行
- quorum：默认值，要求ES中大部分的shard是活跃可用的，才可以执行写操作

(3) 对于读操作，可以设置 replication 为 sync(默认)，这使得操作在主分片和副本分片都完成后才会返回；如果设置replication 为 async 时，也可以通过设置搜索请求参数 _preference 为 primary 来查询主分片，确保文档是最新版本。

六、ES集群如何选举Master节点：

1、Elasticsearch 的分布式原理：

Elasticsearch 会对存储的数据进行切分，划分到不同的分片上，同时每一个分片会生成多个副本，从而保证分布式环境的高可用。ES集群中的节点是对等的，节点间会选出集群的 Master，由 Master 会负责维护集群状态信息，并同步给其他节点。

Elasticsearch 的性能会不会很低：不会，ES只有建立 index 和 type 时需要经过 Master，而数据的写入有一个简单的 Routing 规则，可以路由到集群中的任意节点，所以数据写入压力是分散在整个集群的。

2、ES集群 如何 选举 Master：

Elasticsearch 的选主是 ZenDiscovery 模块负责的，主要包含Ping（节点之间通过这个RPC来发现彼此）和 Unicast（单播模块包含一个主机列表以控制哪些节点需要ping通）这两部分；

- (1) 确认候选主节点的最少投票通过数量（elasticsearch.yml 设置的值 discovery.zen.minimum_master_nodes）
- (2) 选举时，集群中每个节点对所有 master候选节点（node.master: true）根据 nodeId 进行字典排序，然后选出第一个节点（第0位），暂且认为它是master节点。
- (3) 如果对某个节点的投票数达到阈值，并且该节点自己也选举自己，那这个节点就是 master；否则重新选举一直到满足上述条件。

补充：master节点的职责主要包括集群、节点和索引的管理，不负责文档级别的管理；data节点可以关闭http功能。

3、Elasticsearch是如何避免脑裂现象：

(1) 当集群中 master 候选节点数量不小于3个时 (node.master: true) , 可以通过设置最少投票通过数量 (discovery.zen.minimum_master_nodes) , 设置超过所有候选节点一半以上来解决脑裂问题, 即设置为 $(N/2)+1$;

(2) 当集群 master 候选节点 只有两个时, 这种情况是不合理的, 最好把另外一个node.master改成 false。如果我们不改节点设置, 还是套上面的 $(N/2)+1$ 公式, 此时 discovery.zen.minimum_master_nodes应该设置为2。这就出现一个问题, 两个master备选节点, 只要有一个挂, 就选不出master了

七、建立索引阶段性能提升方法：

- (1) 如果是大批量导入, 可以设置 index.number_of_replicas: 0 关闭副本, 等数据导入完成之后再开启副本
- (2) 使用批量请求并调整其大小: 每次批量数据 5-15 MB 大是个不错的起始点。
- (3) 如果搜索结果不需要近实时性, 可以把每个索引的 index.refresh_interval 改到30s
- (4) 增加 index.translog.flush_threshold_size 设置, 从默认的 512 MB 到更大一些的值, 比如 1 GB
- (5) 使用 SSD 存储介质
- (6) 段和合并: Elasticsearch 默认值是 20 MB/s。但如果用的是 SSD, 可以考虑提高到 100-200 MB/s。如果你在做批量导入, 完全不在意搜索, 你可以彻底关掉合并限流。

聊聊：什么是ElasticSearch？

Elasticsearch是一个基于Lucene的全文搜索引擎, 也可称分布式搜索引擎, 用于Java开发的, 且开源, 具有Http Web和无框架Json文档的分布式。

2、ElasticSearch中的集群、节点、索引、文档、类型是什么？

集群：是一个或多个服务器的集合, 共同保存数据并提供所有节点的联合索引和搜索功能。集群有唯一标志, 为"ElasticSearch"。

节点：节点是属于集群一部分的单体服务器, 储存有数据并参与集群索引和搜索功能。如果节点设置为按名称加入集群, 则该节点只能是集群的一部分。

索引：类似关系型数据库中的数据库, 有一个定义多重类型的映射。索引是逻辑名称空间, 可映射到一个或多个主分片, 并且可以有无限个数的副本分片。

文档：文档类似关系型数据库中的数据行, 不同的是处在索引中的文档可以有不同的结构或字段, 但是通用字段应该具有相同的数据类型。

聊聊：列出 10 家使用 Elasticsearch 作为其应用程序的搜索引擎和数据库的公司？

参与过Elastic中文社区活动或者经常关注社区动态的人都知道，使用的公司太多了，列举如下（排名不分先后）：

阿里、腾讯、字节跳动、百度、京东、美团、小米、滴滴、携程、贝壳找房、360、IBM、顺丰快递等等，几乎能想到的互联网公司都在使用Elasticsearch。

聊聊：ElasticSearch中的分片是什么？

谈到分片需要谈到索引，索引是类似关系型数据库中的数据库，有一个定义多重类型的映射。索引是逻辑名称空间，可映射到一个或多个主分片，并且可以有无限个数的副本分片。因此分片是索引被分割成分布在多个节点上的元素。

聊聊：ElasticSearch特点

1、Elasticsearch 是一个分布式的 RESTful 风格的搜索和数据分析引擎

- (1) 查询：搜索方式随心而变。
- (2) 分析：可探索数据的趋势和模式。
- (3) 速度：速度快。
- (4) 可扩展性：个人和企业服务器上都可用。
- (5) 弹性：Elasticsearch 运行在一个分布式的环境中。
- (6) 灵活性：具备多个案例场景，支持所有数据类型
- (7) HADOOP & SPARK：Elasticsearch + Hadoop

2、Elasticsearch是一个高度可伸缩的开源全文搜索和分析引擎。它允许您快速和接近实时地存储、搜索和分析大量数据。

这里有一些使用Elasticsearch的用例：

- (1) 网上商店。
- (2) 分析、调研日志或事务数据。
- (3) 实时调度用户关注信息的推送。
- (4) 结合Kibana (Elasticsearch/ loghide /Kibana堆栈的一部分)来构建自定义仪表板，以可视化自定义数据。此外，还可以使用Elasticsearch聚合功能对数据执行复杂的业务智能查询。

聊聊：谈谈分词与倒排索引的原理

分词：分词用于检索，英文的分词是按单词之间空格区分，中文要考虑效率和准确分词率，防止出现歧义。

倒排：根据文档内容找文档，从关键字去找文档。

倒排索引是搜索引擎的核心。搜索引擎的主要目标是在查找发生搜索条件的文档时提供快速搜索。区别于传统的正向索引，倒排索引会再存储数据时将关键词和数据进行关联，保存到倒排表中，然后查询时，将查询内容进行分词后在倒排表中进行查询，最后匹配数据即可。Elasticsearch 使用一种称为倒排索引的结构，ES中的倒排索引其实就是 **lucene 的倒排索引**，它适用于快速的全文搜索。正向索引（forward index），就是搜索引擎会将待搜索的文件都对应一个文件 ID，搜索时将这个ID 和搜索关键字进行对应，形成 K-V 对，然后对关键字进行统计计数。但是互联网上收录在搜索引擎中的文档的数目是个天文数字，这样的索引结构根本无法满足**实时返回排名结果**的要求。所以，搜索引擎会将正向索引重新构建为反向索引（inverted index，倒排索引），即**把文件ID对应到关键词的映射，转换为关键词到文件ID的映射**，每个关键词都对应着一系列的文件，并保存到倒排表中，查询时会将内容进行分词后在倒排表中进行查询，最后匹配数据即可。这些文件中都出现这个关键词。

聊聊：elasticsearch 的倒排索引是什么

我们传统的检索方式是通过遍历整篇文章，逐个比对找到对应的关键词位置，

而倒排索引是通过分词策略，形成词和文章的关系映射表，这种 *词典+映射表的方式就是倒排索引*，有点类似于我们以前使用的新华字典。倒排索引可极大的提高查询效率。

聊聊：elasticsearch 索引数据多了怎么办，如何调优，部署

- 1.在设计的时候可以基于模板+时间滚动方式创建索引，每天递增数据，避免单个索引很大的情况出现。
- 2.在存储的时候，冷热数据分开存储,比如最近3天的数据作为热数据，其他的作为冷数据，冷数据的话，由于不会再写入新数据了，可以考虑定期force_merge（强制合并）和shrink（压缩）的方式进行处理，节约空间和检索效率
- 3.由于es支持动态扩展，所有可以多加几台机器来缓解集群压力。

聊聊：什么是近实时搜索？

在 Elasticsearch 和磁盘之间是**文件系统缓存**。在内存索引缓冲区中的文档会被写入到一个新的段中。但是这里新段会被先写入到文件系统缓存，这一步代价会比较低，稍后再被刷写到磁盘—这一步代价比较高。**不过只要文件已经在缓存中，就可以像其它文件一样被打开和读取了。**在 Elasticsearch 中，写入和打开一个新段的轻量的过程叫做 refresh。默认情况下每个分片会每秒自动刷新一次，即**刷新文件系统缓存**。这就是为什么我们说 Elasticsearch 是 近实时搜索：文档的变化并不是立即对搜索可见，但会在一秒之内变为可见。这些行为可能会对新用户造成困惑：他们索引了一个文档然后尝试搜索它，但却没有搜到。这个问题的解决办法是用 refresh API 执行一次手动刷新：/users/_refresh。

聊聊：如何理解 Elasticsearch 的近实时的性质，并改善它的不足？

并不是所有的情况都需要每秒刷新。可能你正在使用 Elasticsearch 索引大量的日志文件，你可能想**优化索引速度**而不是近实时搜索，可以通过设置 refresh_interval，降低每个索引的刷新频率。refresh_interval 可以在既存索引上进行动态更新。在生产环境中，当你正在建立一个大的新索引时，可以先关闭自动刷新，待开始使用该索引时，再把它们调回来。

```
# 关闭自动刷新
PUT /users/_settings
{ "refresh_interval": -1 }

# 每一秒刷新
PUT /users/_settings
{ "refresh_interval": "1s" }
```

聊聊：Elasticsearch 中索引在设计阶段如何调优？

- 1) 根据业务增量需求，采取基于日期模板创建索引，通过roll over API滚动索引；
- 2) 使用别名进行索引管理；
- 3) 每天凌晨定时对索引做force_merge操作，以释放空间；
- 4) 采取冷热分离机制，热数据存储到SSD，提高检索效率；冷数据定期进行shrink操作，以缩减存储；
- 5) 采取curator进行索引的生命周期管理；
- 5) 仅针对需要分词的字段，合理的设置分词器；
- 6) Mapping阶段充分结合各个字段的属性，是否需要检索、是否需要存储等。

聊聊：解释一下 Elasticsearch Node？

节点是Elasticsearch的实例。实际业务中，会说：ES集群包含3个节点、7个节点。

这里节点实际就是：一个独立的Elasticsearch进程，一般将一个节点部署到一台独立的服务器或者虚拟机、容器中。

不同节点根据角色不同，可以划分为：

主节点

帮助配置和管理在整个集群中添加和删除节点。

数据节点

存储数据并执行诸如CRUD（创建/读取/更新/删除）操作，对数据进行搜索和聚合的操作。

- 1、客户端节点（或者说：协调节点） 将集群请求转发到主节点，将与数据相关的请求转发到数据节点
- 2、摄取节点

用于在索引之前对文档进行预处理。

聊聊：Elasticsearch 中分析器由哪几部分组成？

分析器由三部分构成：

- 1、字符过滤器 (Character Filters)
- 2、分词器 (Tokenizers)
- 3、分词过滤器 (Token Filters)

一个分析器不一定这三个部分都有，但是一般会包含分词器。ES自带的分析器有如下几种：

Standard Analyzer、Simple Analyzer、Whitespace Analyzer、Stop Analyzer、Keyword Analyzer、Pattern Analyzer、Language Analyzers 和 Fingerprint Analyzer。

Elasticsearch内置了若干分析器类型，其中常用的是标准分析器，叫做“standard”。其中Standard Analyzer是ES默认的分析器，如果没有指定任何分析器的话，ES将默认使用这种分析器。

分析器 (Analyzer) 通常由一个Tokenizer (怎么分词)，以及若干个TokenFilter (过滤分词)、Character Filter (过滤字符) 组成。

聊聊：Elasticsearch中的 Ingest 节点如何工作？

ingest节点可以看作是数据前置处理转换的节点，支持pipeline管道设置，可以使用ingest对数据进行过滤、转换等操作，类似于logstash中filter的作用，功能相当强大。

聊聊：Elasticsearch 中的分析器是什么？

分析 (analysis) 机制用于进行全文文本 (Full Text) 的分词，以建立供搜索用的反向索引。

1、在ElasticSearch中索引数据时，数据由为索引定义的Analyzer在内部进行转换。分析器由一个Tokenizer和零个或多个TokenFilter组成。

编译器可以在一个或多个CharFilter之前。分析模块允许您在逻辑名称下注册分析器，然后可以在映射定义或某些API中引用它们。

2、Elasticsearch附带了许多可以随时使用的预建分析器。或者可以组合内置的字符过滤器，编译器和过滤器来创建自定义分析器。

聊聊：Elasticsearch 支持哪些配置管理工具？

Ansible、Chef、Puppet和Salt Stack是DevOps团队使用的Elasticsearch支持的配置工具。

聊聊：ElasticSearch 的节点类型有什么区别？

节点是指ElasticSearch的实例。当启动Elasticsearch的实例，就会启动至少一个节点。

相同集群名的多个节点的连接就组成了一个集群，在默认情况下，集群中的每个节点都可以处理http请求和集群节点间的数据传输，集群中所有的节点都知道集群中其他所有的节点，可以将客户端请求转发到适当的节点。

节点有以下类型：

主(master)节点：在一个节点上node.master设置为True（默认）的时候，它有资格被选作为主节点，控制整个集群。

数据(data)节点：在一个节点上node.data设置为True（默认）的时候。该节点保存数据和执行数据相关的操作，如增删改查，搜索，和聚合。

客户端节点：当一个节点的node.master和node.data都设置为false的时候，它既不能保持数据也不能成为主节点，该节点可以作为客户端节点，可以响应用户的情况，并把相关操作发送到其他节点。

部落节点：当一个节点配置tribe.*的时候，它是一个特殊的客户端，它可以连接多个集群，在所有连接的集群上执行搜索和其他操作。

聊聊：Elasticsearch了解多少，聊聊你们公司ES的集群架构，索引数据大小，分片有多少，以及一些调优手段。

面试官：

想了解应聘者之前公司接触的ES使用场景、规模，有没有做过比较大规模的索引设计、规划、调优。

解答策略：

如实结合自己的实践场景回答即可。

比如：ES集群架构13个节点，索引根据通道不同共20+索引，根据日期，每日递增20+

索引：10分片，每日递增1亿+数据，每个通道每天索引大小控制：150GB之内。

索引层面调优手段：

1、设计阶段调优

- 1) 根据业务增量需求，采取基于日期模板创建索引，通过roll over API滚动索引；
- 2) 使用别名进行索引管理；
- 3) 每天凌晨定时对索引做force_merge操作，以释放空间；
- 4) 采取冷热分离机制，热数据存储到SSD，提高检索效率；冷数据定期进行shrink操作，以缩减存储；
- 5) 采取curator进行索引的生命周期管理；
- 6) 仅针对需要分词的字段，合理的设置分词器；
- 7) Mapping阶段充分结合各个字段的属性，是否需要检索、是否需要存储等。

2、写入调优

- 1) 写入前副本数设置为0；
- 2) 写入前关闭refresh_interval设置为-1，禁用刷新机制；
- 3) 写入过程中：采取bulk批量写入；
- 4) 写入后恢复副本数和刷新间隔；
- 5) 尽量使用自动生成的id。

2.自动生成document id

1. post /index/type

2. 自动生成的id，长度为20个字符，URL安全，base64编码，GUID，分布式系统并行生成时不可能会发生冲突



```
console
1 GET test_index/test_type/1
2
3 PUT /test_index/test_type/1
4 {
5   "test_content": "test test"
6 }
7
8 PUT test_index/test_type/2
9 {
10  "test_content": "my test"
11 }
12
13 POST test_index/test_type
14 {
15   "test_content": "my test"
16 }
17
18
19 {
20   "_index": "test_index",
21   "type": "test_type",
22   "_id": "AW1J04cVHfaC6FwX_5f4",
23   "_version": 1,
24   "result": "created",
25   "_shards": {
26     "total": 2,
27     "successful": 1,
28     "failed": 0
29   },
30   "created": true
31 }
```

6) 当写入数据时，确保bulk请求时轮询访问所有节点。不要发送所有请求到一个节点，导致这一个节点要在内存存储所有请求的数据去处理

3、查询调优

1、filesystem cache越大越好 为了使得搜索速度更快，es严重依赖filesystem cache

一般来说，需要至少一半的 可用内存作为filesystem cache，这样es可以在物理内存中 保有 索引的热点区域 (hot regions of the index)

2、用更好的硬件 搜索一般是I/O bound的，此时，你需要

1. 为filesystem cache分配更多的内存
2. 使用SSD硬盘
3. 使用local storage (不要使用NFS、SMB 等remote filesystem)
4. 亚马逊的 弹性块存储 (Elastic Block Storage) 也是极好的，当然，和local storage比起来，它还是要慢点 如果你的搜索是 CPU-bound，买好的CPU吧

3、文档模型 (document modeling) 文档需要使用合适的类型，从而使得 search-time operations 消耗更少的资源。咋作呢？

答：避免 join操作。具体是指 a.nested 会使得查询慢 好几倍 b.parent-child关系 更是使得查询慢几百倍 如果 无需join 能解决问题，则查询速度会快很多

4、预索引 数据 根据“搜索数据最常用的方式”来最优化索引数据的方式

举个例子：所有文档都有price字段，大部分query 在 fixed ranges 上运行 range aggregation。你可以把给定范围的数据 预先索引下。然后，使用 terms aggregation

5、Mappings (能用 keyword 最好了) 数字类型的数据，并不意味着一定非得使用numeric类型的字段。

一般来说，存储标识符的 字段 (书号ISBN、或来自数据库的 标识一条记录的 数字)，使用keyword更好 (integer, long 不好哦)

6、避免运行脚本 一般来说，脚本应该避免。如果他们是绝对需要的，你应该使用painless和expressions引擎。

7、搜索rounded 日期 日期字段上使用now，一般来说不会被缓存。但，rounded date则可以利用上query cache rounded到分钟等

8、强制merge只读的index 只读的index可以从“merge成 一个单独的 大segment”中收益

9、预热 全局序数 (global ordinals) 。全局序数用于在keyword字段上运行terms aggregations。es不知道哪些fields将用于/不用于 term aggregation

因此全局序数在需要时才加载进内存，但可以在mapping type上，定义 eager_global_ordinals==true。这样，refresh时就会加载 全局序数

10、预热 filesystem cache 机器重启时，filesystem cache就被清空。

OS将index的热点区域 (hot regions of the index) 加载进filesystem cache是需要花费一段时间的。设置 index.store.preload 可以告知OS 这些文件需要提早加载进入内存

11、使用索引排序来加速连接 索引排序对于以较慢的索引为代价来加快连接速度非常有用。在索引分类文档中阅读更多关于它的信息。

12、使用preference来优化高速缓存利用率 有多个缓存可以帮助提高搜索性能，例如文件系统缓存，请求缓存或查询缓存。

然而，所有这些缓存都维护在节点级别，这意味着如果连续运行两次相同的请求，则有一个或多个副本，并使用循环（默认路由算法），那么这两个请求将转到不同的分片副本，阻止节点级别的缓存帮助。

由于搜索应用程序的用户一个接一个地运行类似的请求是常见的，例如为了分析索引的较窄的子集，使用标识当前用户或会话的优选值可以帮助优化高速缓存的使用。

13、副本可能有助于吞吐量，但不会一直存在 除了提高弹性外，副本可以帮助提高吞吐量。例如，如果您有单个分片索引和三个节点，则需要将副本数设置为2，以便共有3个分片副本，以便使用所有节点。

现在假设你有一个2-shards索引和两个节点。

在一种情况下，副本的数量是0，这意味着每个节点拥有一个分片。在第二种情况下，副本的数量是1，这意味着每个节点都有两个碎片。

哪个设置在搜索性能方面表现最好？通常情况下，每个节点的碎片数少的设置将会更好。

原因在于它将可用文件系统缓存的份额提高到了每个碎片，而文件系统缓存可能是Elasticsearch的1号性能因子。

同时，要注意，没有副本的设置发生在单个节点故障的情况下会出现故障，因此在吞吐量和可用性之间进行权衡。

那么复制品的数量是多少？如果您有一个具有num_nodes节点的群集，那么num primaries总共是主分片，如果您希望能够一次处理max_failures节点故障，那么正确的副本数是 $\max(\max_failures, \text{ceil}(\text{num_nodes} / \text{num_primaries}) - 1)$ 。

14、打开自适应副本选择 当存在多个数据副本时，elasticsearch可以使用一组称为自适应副本选择的标准，根据包含分片的每个副本的节点的响应时间，服务时间和队列大小来选择数据的最佳副本。这可以提高查询吞吐量并减少搜索量大的应用程序的延迟。

4、其他调优

部署调优，业务调优等。

上面的提及一部分，面试官就基本对你之前的实践或者运维经验有所评估了。

聊聊：客户端在和ES集群连接时，如何选择特定的节点执行请求的？

客户端是通过transport 模块远程连接一个 elasticsearch 集群。它并不加入到集群中，只是简单的获得一个或者多个初始化的 transport 地址，并以 轮询 的方式与这些地址进行通信。

聊聊：Elasticsearch 中，是怎么根据一个词找到对应的倒排索引的？

首先我们应该了解一下什么是倒排索引，倒排索引就是通过分词策略把分词和文章形成一个关系映射表，这种词典和映射表的方式就是我们的倒排索引，所以当我们去检索一个词语的时候，会根据文档id去整个索引库中去查询到匹配的索引，然后返回给客户端。

聊聊：Elasticsearch的倒排索引与Mysql正排索引的区别？

一切设计都是为了提高搜索的性能

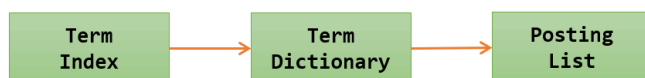
倒排索引 (Inverted Index) 也叫反向索引，有反向索引必有正向索引。通俗地来讲，正向索引是通过key找value，反向索引则是通过value找key。

先来回忆一下我们是怎么插入一条索引记录的：

```
curl -X PUT "localhost:9200/user/_doc/1" -H 'Content-Type: application/json' -d '{
  "name" : "Jack",
  "gender" : 1,
  "age" : 20
}
```

其实就是直接PUT一个JSON的对象，这个对象有多个字段，在插入这些数据到索引的同时，Elasticsearch还为这些字段建立索引——倒排索引，因为Elasticsearch最核心功能是搜索。

那么，倒排索引是个什么样子呢？



首先，来搞清楚几个概念，为此，举个例子：

假设有个user索引，它有四个字段：分别是name, gender, age, address。

画出来的话，大概是下面这个样子，跟关系型数据库一样

id	name	gender	age	address
1	张三	1	22	北京市朝阳区
2	李四	2	21	上海市徐汇区
3	王五	1	23	上海市虹口区

Term (单词)：一段文本经过分析器分析以后就会输出一串单词，这一个个的就叫做Term（直译为：单词）

Term Dictionary (单词字典)：顾名思义，它里面维护的是Term，可以理解为Term的集合

Term Index (单词索引)：为了更快的找到某个单词，我们为单词建立索引

Posting List (倒排列表)：倒排列表记录了出现过某个单词的所有文档的文档列表及单词在该文档中出现的位置信息，每条记录称为一个倒排项(Posting)。根据倒排列表，即可获知哪些文档包含某个单词。（PS：实际的倒排列表中并不只是存了文档ID这么简单，还有一些其它的信息，比如：词频（Term出现的次数）、偏移量（offset）等，可以想象成是Python中的元组，或者Java中的对象）

（PS：如果类比现代汉语词典的话，那么Term就相当于词语，Term Dictionary相当于汉语词典本身，Term Index相当于词典的目录索引）

我们知道，每个文档都有一个ID，如果插入的时候没有指定的话，Elasticsearch会自动生成一个，因此ID字段就不多说了

上面的例子，Elasticsearch建立的索引大致如下：

name字段：

Term	Posting List
张三	1
李四	2
王五	3

age字段：

Term	Posting List
21	2
22	1
23	3

gender字段：

Term	Posting List
1	[1, 3]
2	2

address字段：

Term	Posting List
北京市	1
上海市	[2, 3]
徐汇区	2
虹口区	3
朝阳区	1

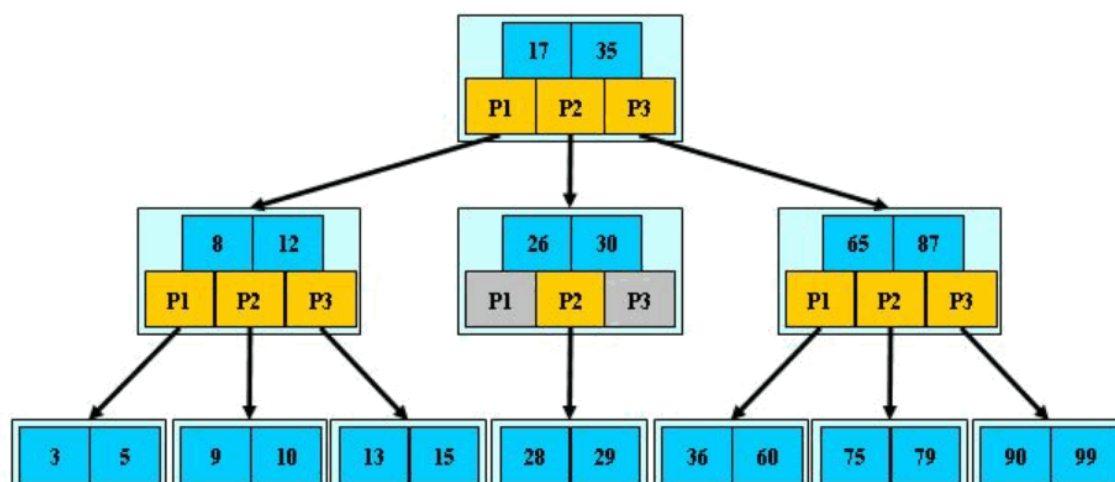
Elasticsearch分别为每个字段都建立了一个倒排索引。

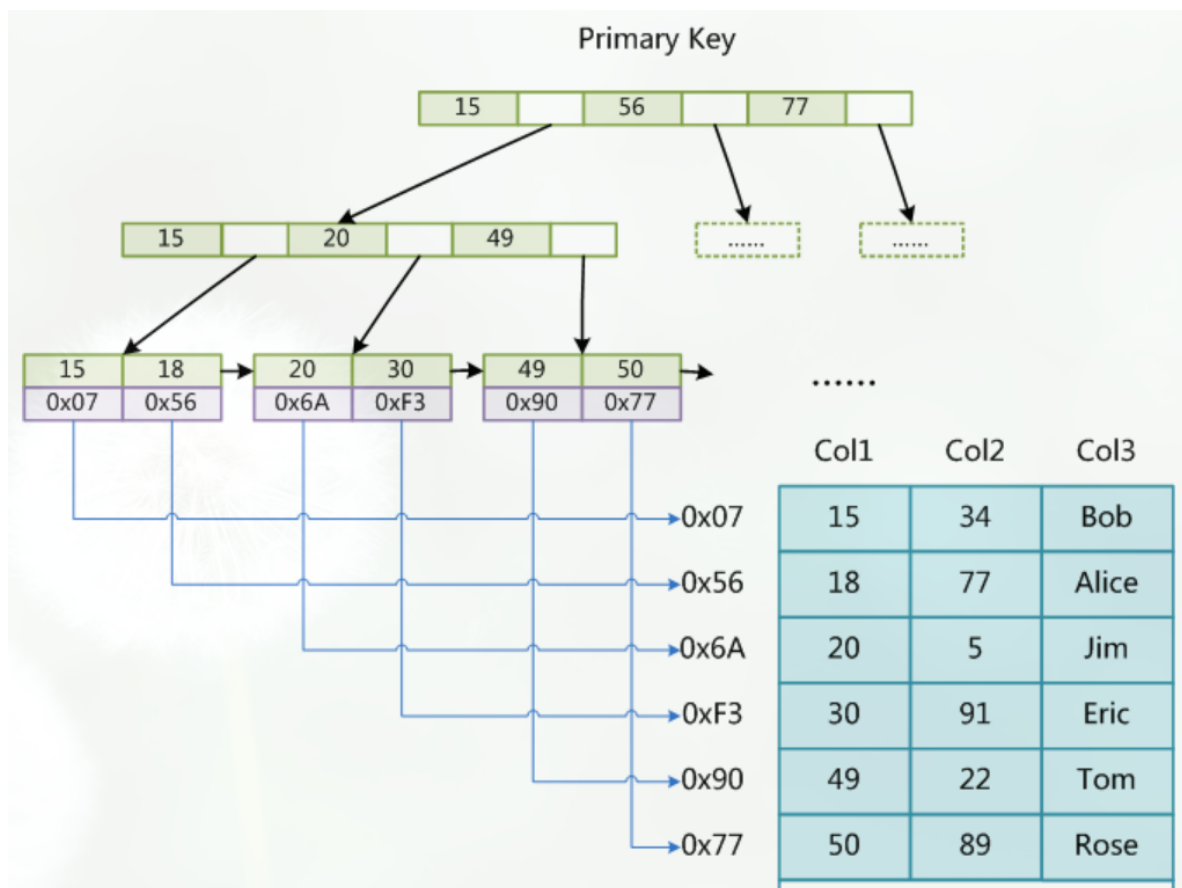
比如，在上面“张三”、“北京市”、22 这些都是Term，而[1, 3]就是Posting List。Posting list就是一个数组，存储了所有符合某个Term的文档ID。

只要知道文档ID，就能快速找到文档。可是，要怎样通过我们给定的关键词快速找到这个Term呢？

当然是建索引了，为Terms建立索引，最好的就是B-Tree索引（PS：MySQL就是B树索引最好的例子）。

首先，让我们来回忆一下MyISAM存储引擎中的索引是什么样的：



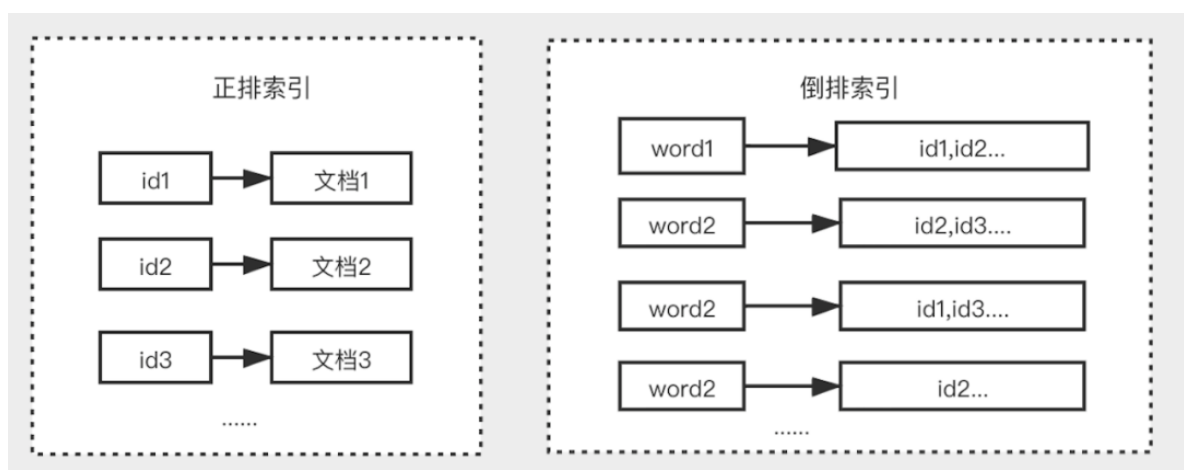


我们查找Term的过程跟在MyISAM中记录ID的过程大致是一样的

MyISAM中，索引和数据是分开，通过索引可以找到记录的地址，进而可以找到这条记录

倒排索引是区别于正排索引的概念：

- **正排索引**：是以文档对象的唯一 ID 作为索引，以文档内容作为记录。
- **倒排索引**：Inverted index，指的是将文档内容中的单词作为索引，将包含该词的文档 ID 作为记录。

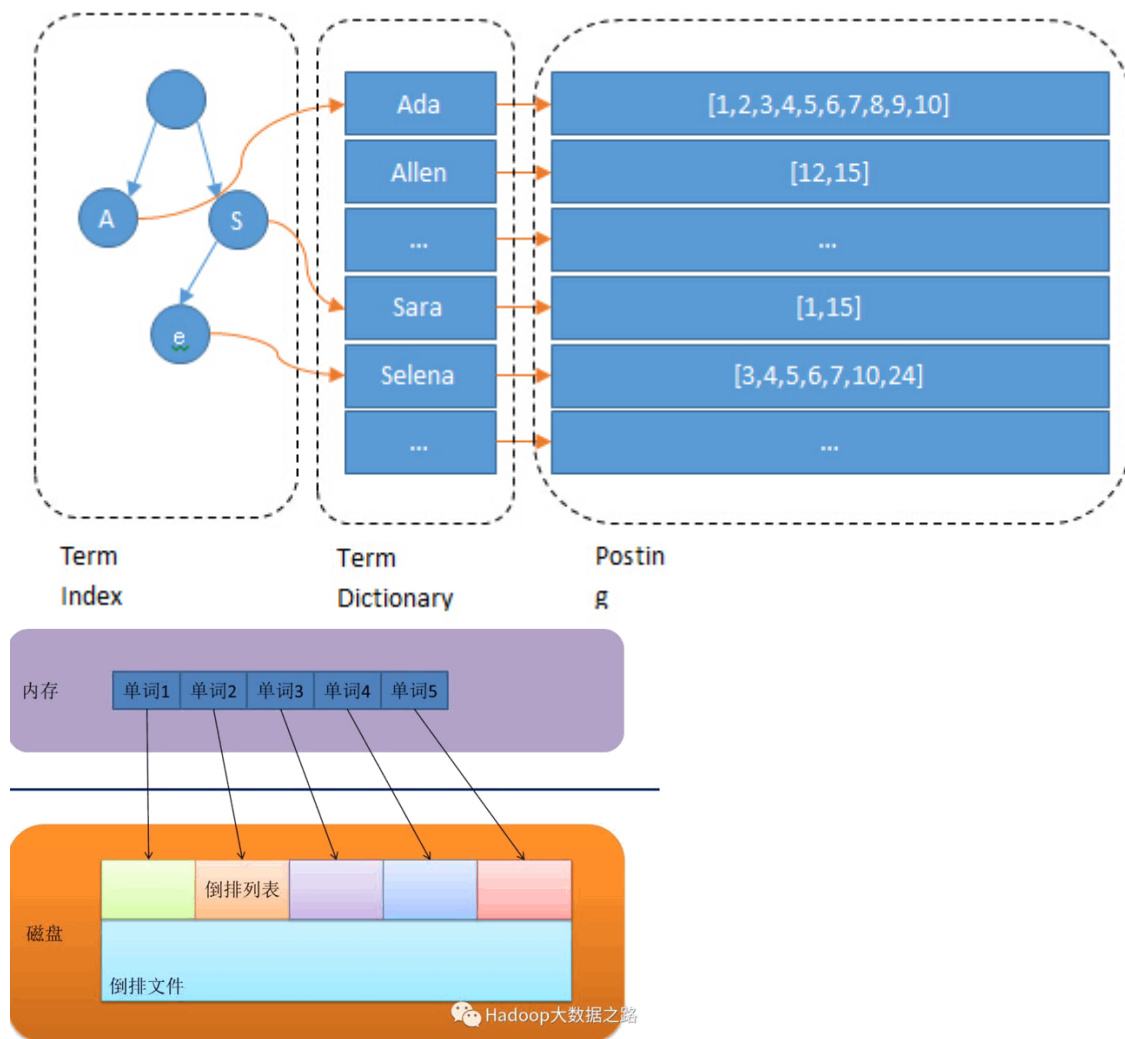


在倒排索引中，通过Term索引可以找到Term在Term Dictionary中的位置，进而找到Posting List，有了倒排列表就可以根据ID找到文档了

(PS：可以这样理解，类比MyISAM的话，Term Index相当于索引文件，Term Dictionary相当于数据文件)

(PS: 其实, 前面我们分了三步, 我们可以把Term Index和Term Dictionary看成一步, 就是找Term。因此, 可以这样理解倒排索引: 通过单词找到对应的倒排列表, 根据倒排列表中的倒排项进而可以找到文档记录)

为了更进一步理解, 下面从网上搞了两张图来具现化这一过程:



倒排索引的生成过程

下面通过一个例子来说明下倒排索引的生成过程。

假设目前有以下两个文档内容:

苏州街维亚大厦
桔子酒店苏州街店

其处理步骤如下:

1、正排索引给每个文档进行编号, 作为其唯一的标识。

文档 id	content
1	苏州街维亚大厦
2	桔子酒店苏州街店

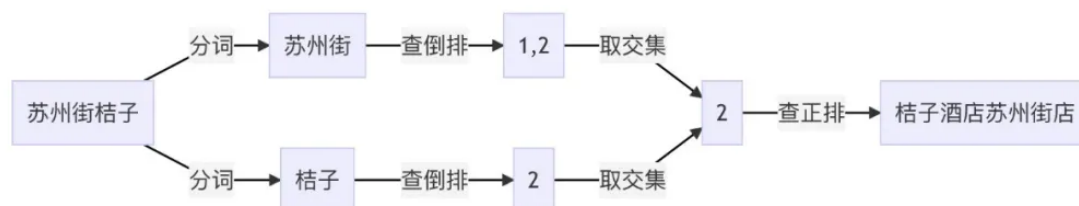
2、生成倒排索引:

- 首先要对字段的内容进行分词，分词就是将一段连续的文本按照语义拆分为多个单词，这里两个文档包含的关键词有：苏州街、维亚大厦...
- 然后按照单词来作为索引，对应的文档 id 建立一个链表，就能构成上述的倒排索引结构。

Word	文档 id
苏州街	1,2
维亚大厦	1
维亚	1
桔子	2
酒店	2
大赛	1

有了倒排索引，能快速、灵活地实现各类搜索需求。整个搜索过程中我们不需要做任何文本的模糊匹配。

例如，如果需要在上述两个文档中查询 苏州街桔子，可以通过分词后 苏州街 查到 1、2，通过 桔子 查到 2，然后再进行取交集等操作得到最终结果。



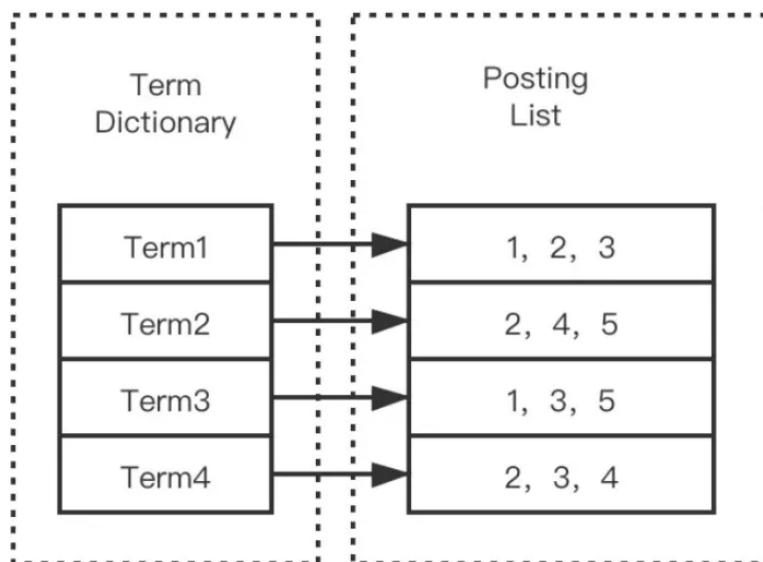
倒排索引的结构

根据倒排索引的概念，我们可以用一个 Map 来简单描述这个结构。这个 **Map 的 Key 的即是分词后的单词**，这里的单词称为 Term，一系列的 Term 组成了倒排索引的第一个部分 —— Term Dictionary (索引表，可简称为 Dictionary)。

倒排索引的另一部分为 Postings List (记录表)，也对应上述 **Map 结构的 Value 部分集合**。

记录表由所有的 Term 对应的数据 (Postings) 组成，它不仅仅为文档 id 信息，可能包含以下信息：

- **文档 id (DocId, Document Id)**，包含单词的所有文档唯一 id，**用于去正排索引中查询原始数据**。
- **词频 (TF, Term Frequency)**，记录 Term 在每篇文档中出现的次数，**用于后续相关性算分**。
- **位置 (Position)**，记录 Term 在每篇文档中的分词位置 (多个)，**用于做词语搜索 (Phrase Query)**。
- **偏移 (Offset)**，记录 Term 在每篇文档的开始和结束位置，**用于高亮显示等**。



Lucene 倒排索引实现

全文搜索引擎在海量数据的情况下是需要存储大量的文本，所以面临以下问题：

- Dictionary 是比较大的（比如我们搜索中的一个字段可能有上千万个 Term）
- Postings 可能会占据大量的存储空间（一个Term多的有几百万个doc）

因此上面说的基于 Map 的实现方式几乎是不可行的。

在海量数据背景下，倒排索引的实现直接关系到存储成本以及搜索性能。

为此，Lucene 引入了多种巧妙的数据结构和算法。其倒排索引实现拥有以下特性：

- 以较低的存储成本存储在磁盘（索引大小大约为被索引文本的20-30%）
- 快速读写

下面将根据倒排索引的结构，按 Posting List 和 Terms Dictionary 两部分来分析 Lucene 中的实现。

Posting List 实现

PostingList 包含文档 id、词频、位置等多个信息，这些数据之间本身是相对独立的，因此 Lucene 将 Postings List 被拆成三个文件存储：

- **doc后缀文件**：记录 Postings 的 docId 信息和 Term 的词频
- **pay后缀文件**：记录 Payload 信息和偏移量信息
- **pos后缀文件**：记录位置信息

基本所有的查询都会用 .doc 文件获取文档 id，且一般的查询仅需要用到 .doc 文件就足够了，只有对于近似查询等位置相关的查询则需要用位置相关数据。

三个文件整体实现差不多，这里以.doc 文件为例分析其实现。

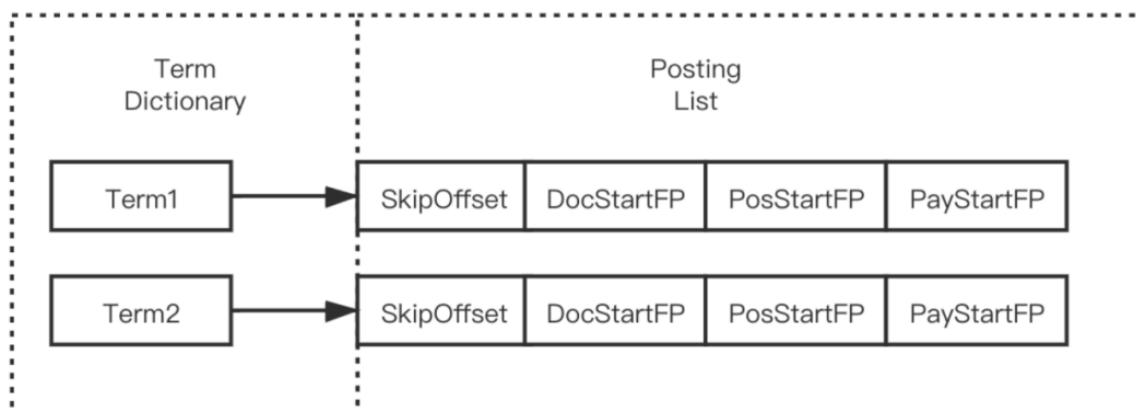
.doc 文件存储的是每个 Term 对应的文档 Id 和词频。每个 Term 都包含一对 TermFreqs 和 SkipData 结构。

其中 TermFreqs 存放 docId 和词频信息，SkipData 为跳表信息，用于实现 TermFreqs 内部的快速跳转。



Term Dictionary 实现

Terms Dictionary（索引表）存储所有的 Term 数据，同时它也是 Term 与 Postings 的关系纽带，存储了每个 Term 和其对应的 Postings 文件位置指针。



聊聊：Elasticsearch索引数据多了怎么办，如何调优，部署？

答：

面试官：想了解大数据量的运维能力。

解答：索引数据的规划，应在前期做好规划，正所谓“设计先行，编码在后”，这样才能有效的避免突如其来的数据激增导致集群处理能力不足引发的线上客户检索或者其他业务受到影响。

如何调优，正如问题1所说，这里细化一下：

3.1 动态索引层面

基于模板+时间+rollover api滚动创建索引，举例：设计阶段定义：blog索引的模板格式为：blog_index_时间戳的形式，每天递增数据。

这样做的好处：不至于数据量激增导致单个索引数据量非常大，接近于上线2的32次幂-1，索引存储达到了TB+甚至更大。

一旦单个索引很大，存储等各种风险也随之而来，所以要提前考虑+及早避免。

3.2 存储层面

冷热数据分离存储，热数据（比如最近3天或者一周的数据），其余为冷数据。对于冷数据不会再写入新数据，可以考虑定期 force_merge 加 shrink 压缩操作，节省存储空间和检索效率。

3.3 部署层面

一旦之前没有规划，这里就属于应急策略。结合ES自身的支持动态扩展的特点，动态新增机器的方式可以缓解集群压力，注意：如果之前主节点等规划合理，不需要重启集群也能完成动态新增的。

聊聊：Elasticsearch是如何实现master选举的？

答：

面试官：想了解ES集群的底层原理，不再只关注业务层面了。

解答：

前置前提：

1、只有候选主节点（master: true）的节点才能成为主节点。

2、最小主节点数（min_master_nodes）的目的是防止脑裂。

这个我看了各种网上分析的版本和源码分析的书籍，云里雾里。

核对了一下代码，核心入口为findMaster，选择主节点成功返回对应Master，否则返回null。

选举流程大致描述如下：

第一步：确认候选主节点数达标，elasticsearch.yml设置的值

discovery.zen.minimum_master_nodes;

第二步：比较：先判定是否具备master资格，具备候选主节点资格的优先返回；若两节点都为候选主节点，则id小的值会主节点。注意这里的id为string类型。

聊聊：Elasticsearch 的 master 选举流程？

- Elasticsearch 的选主是 ZenDiscovery 模块负责的，主要包含 **Ping**（节点之间通过这个 RPC 来发现彼此）和 **Unicast**（单播模块包含一个主机列表以控制哪些节点需要 ping 通）这两部分
- 对所有可以成为 master 的节点（node.master: true）根据 nodeId 字典排序，每次选举每个节点都把自己所知道节点排一次序，然后选出第一个（第 0 位）节点，暂且认为它是 master 节点。
- 如果对某个节点的投票数达到一定的值（**可以成为 master 节点数 $n/2+1$** ）并且该节点自己也选举自己，那这个节点就是 master。否则重新选举一直到满足上述条件。
- master 节点的职责主要包括集群、节点和索引的管理，**不负责文档级别的管理**；data 节点可以关闭 http 功能。

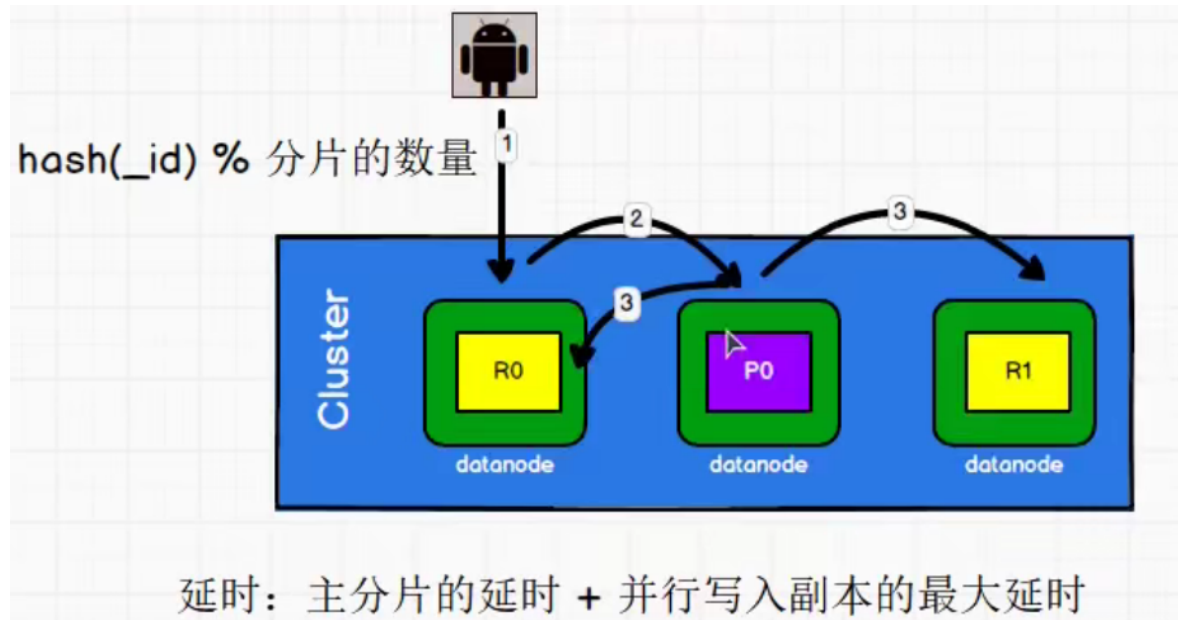
聊聊：详细描述一下Elasticsearch索引文档的过程。

面试官：想了解ES的底层原理，不再只关注业务层面了。

解答：

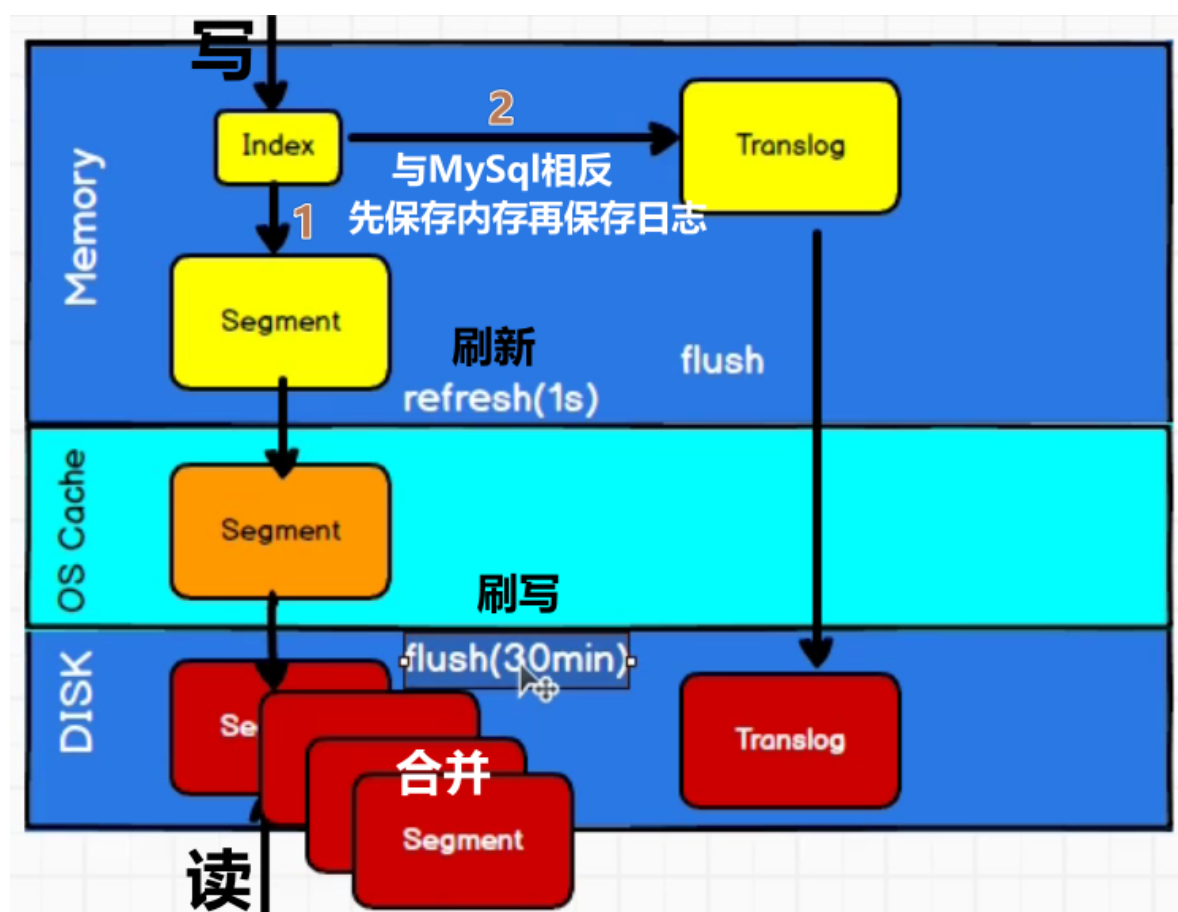
首先客户端向集群发出索引文档的请求，它会选择**任何一个节点**，

这个节点当接收到请求后会根据路由算法找到应该放的那个主分片的位置，从而索引数据，



之后为了保证数据的完整性，它会将它的副本数据进行同步，同步完成后客户端就可以进行访问了。

细节方面：



用户的索引请求发过来之后，首先协调节点默认使用文档ID参与哈希计算（也支持通过routing），

```
shard = hash(document_id) % (num_of_primary_shards)
```

即

分片位置索引 = 将文档ID或路由ID进行哈希计算后的值 % 所有分片总数

随后会在内存（memory）中建立一个索引（Index），这个Index会在内存中形成一个分段对象（Segment），

为了防止数据出现问题，会同时在索引数据之后写入到日志（Translog）当中，

在此过程中，每隔1秒钟，会向Segment会将数据刷新到系统文件缓存区（OS Cache），以方便接收用户的查询，

因为如果让用户查询直接访问内存或磁盘，会使速度变慢。

当过了30分钟或者Translog中的数据超过了512M，Os Cache中的Segment会将数据刷写（flush）到磁盘当中，刷写后内存中的缓冲将被清除。

此时一旦刷写的数据比较多了的话（磁盘中有多个Segment），磁盘就会将这些分段进行合并。

聊聊：详细描述一下Elasticsearch索引文档的过程。

答：

协调节点默认使用文档ID参与计算（也支持通过routing），以便为路由提供合适的分片。

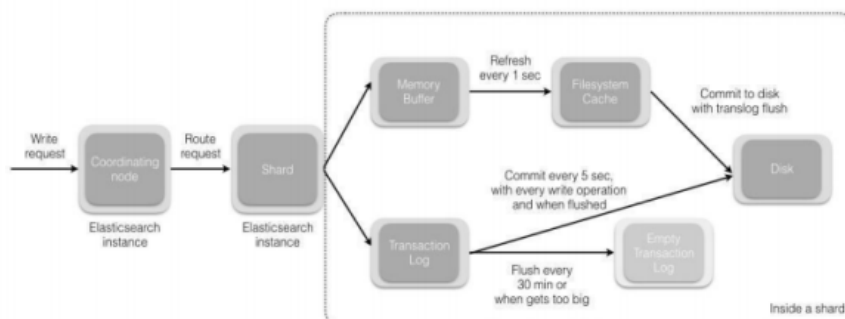
```
shard = hash(document_id) % (num_of_primary_shards)
```

1、当分片所在的节点接收到来自协调节点的请求后，会将请求写入到Memory Buffer，然后定时（默认是每隔1秒）写入到 Filesystem Cache，这个从Memory Buffer到Filesystem Cache的过程就叫做refresh；

2、当然在某些情况下，存在Memory Buffer和Filesystem Cache的数据可能会丢失，ES是通过translog的机制来保证数据的可靠性的。其实现机制是接收到请求后，同时也会写入到translog中，当Filesystem cache中的数据写入到磁盘中时，才会清除掉，这个过程叫做flush；

3、在 flush 过程中，内存中的缓冲将被清除，内容被写入一个新段，段的fsync将创建一个新的提交点，并将内容刷新到磁盘，旧的translog将被删除并开始一个新的translog。

4、flush触发的时机是定时触发（默认30分钟）或者translog变得太大（默认为512M）时



https://blog.csdn.net/m0_48795607

补充：关于Lucene 的 Segment：

1、Lucene索引是由多个段组成，段本身是一个功能齐全的倒排索引。

2、段是不可变的，允许Lucene将新的文档增量地添加到索引中，而不用从头重建索引。

3、对于每一个搜索请求而言，索引中的所有段都会被搜索，并且每个段会消耗CPU的时钟周、文件句柄和内存。这意味着段的数量越多，搜索性能会越低。

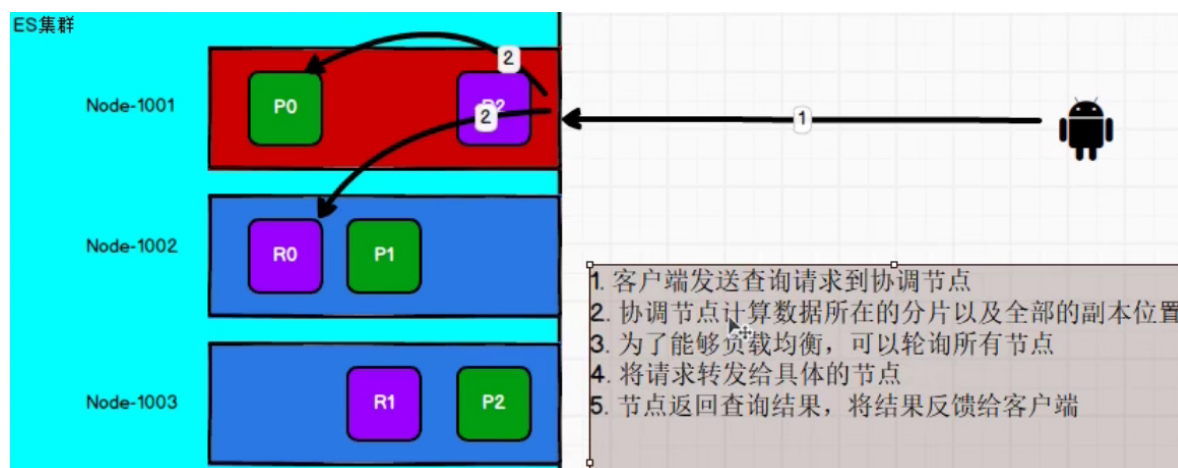
4、为了解决这个问题，Elasticsearch会合并小段到一个较大的段，提交新的合并段到磁盘，并删除那些旧的小段。

###

聊聊：详细描述一下Elasticsearch搜索的过程？

面试官：想了解ES搜索的底层原理，不再只关注业务层面了。

解答：



- 搜索被执行成一个两阶段过程，我们称之为 Query Then Fetch（查询后取回）；
- 在初始查询阶段时，查询会广播到索引中每一个分片拷贝（主分片或者副本分片）。每个分片在本
地执行搜索并构建一个匹配文档的大小为 from + size 的优先队列。PS：在搜索的时候会查询
Filesystem Cache 的，但是有部分数据**还在 Memory Buffer**，所以搜索是近实时的。
- 每个分片返回各自优先队列中 所有文档的 ID 和排序值 给 协调节点，它合并这些值到自己的优先
队
列中来产生一个全局排序后的结果列表。
- 接下来就是取回阶段，协调节点辨别出哪些文档需要被取回，并向相关的分片提交多个 GET 请
求。每
个分片加载并丰富文档，如果有需要的话，接着返回文档给协调节点。一旦所有的文档都被取回
了，
协调节点返回结果给客户端。
- Query Then Fetch 的搜索类型在文档相关性打分的时候参考的是本分片的数据，这样在文档数量
较少
的时候可能不够准确，DFS Query Then Fetch 增加了一个预查询的处理，询问 Term 和
Document
frequency，这个评分更准确，但是性能会变差。

聊聊：详细描述一下Elasticsearch搜索的过程。

答：

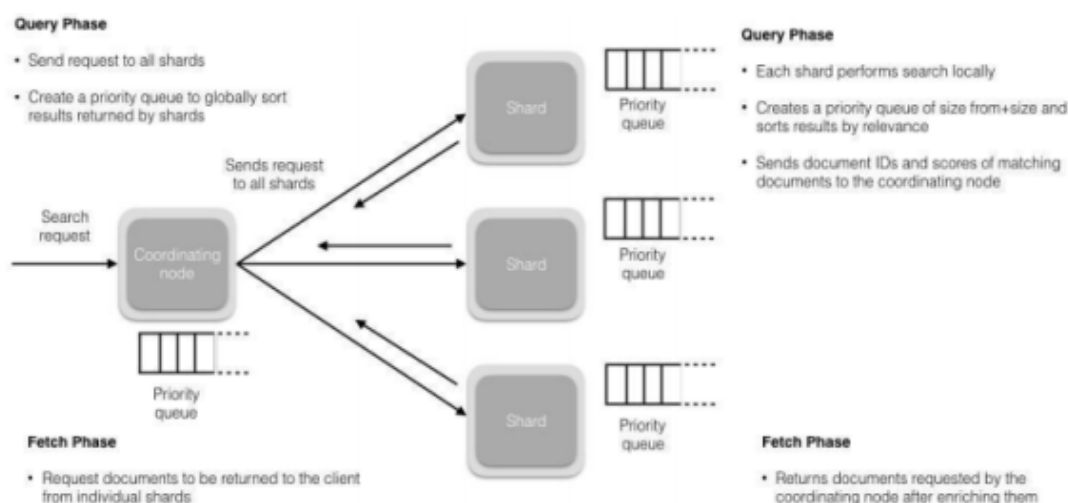
- 1、搜索被执行成一个两阶段过程，我们称之为Query Then Fetch；
- 2、在初始查询阶段时，查询会广播到索引中每一个分片拷贝（主分片或者副本分片）。每个分片在本
地执行搜索并构建一个匹配文档的大小为from + size的优先队列。

PS：在搜索的时候会查询Filesystem Cache的，但是有部分数据还在MemoryBuffer，所以搜索是近实时的。

3、每个分片返回各自优先队列中 所有文档的ID和排序值 给协调节点，它合并这些值到自己的优先队列中来产生一个全局排序后的结果列表。

4、接下来就是 取回阶段，协调节点辨别出哪些文档需要被取回并向相关的分片提交多个GET请求。每个分片加载并 丰富 文档，如果有需要的话，接着返回文档给协调节点。一旦所有的文档都被取回了，协调节点返回结果给客户端。

5、补充：Query Then Fetch的搜索类型在文档相关性打分的时候参考的是本分片的数据，这样在文档数量较少的时候可能不够准确，DFS Query Then Fetch增加了一个预查询的处理，询问Term 和 Document frequency，这个评分更准确，但是性能会变差。



###

聊聊：详细描述一下Elasticsearch更新和删除文档的过程。

答：

1、删除和更新也都是写操作，但是Elasticsearch 中的文档是不可变的，因此不能被删除或者改动以展示其变更；

2、磁盘上的每个段都有一个相应的del文件。当删除请求发送后，文档并没有真的被删除，而是在del文件中被标记为删除。该文档依然能匹配查询，但是会在结果中被过滤掉。当段合并时，在del文件中被标记为删除的文档将不会被写入新段。

3、在新的文档被创建时，Elasticsearch会为该文档指定一个版本号，当执行更新时，旧版本的文档在del文件中被标记为删除，新版本的文档被索引到一个新段。旧版本的文档依然能匹配查询，但是会在结果中被过滤掉。

###

聊聊：Elasticsearch在部署时，对Linux的设置有哪些优化方法？

面试官：想了解对ES集群的运维能力。

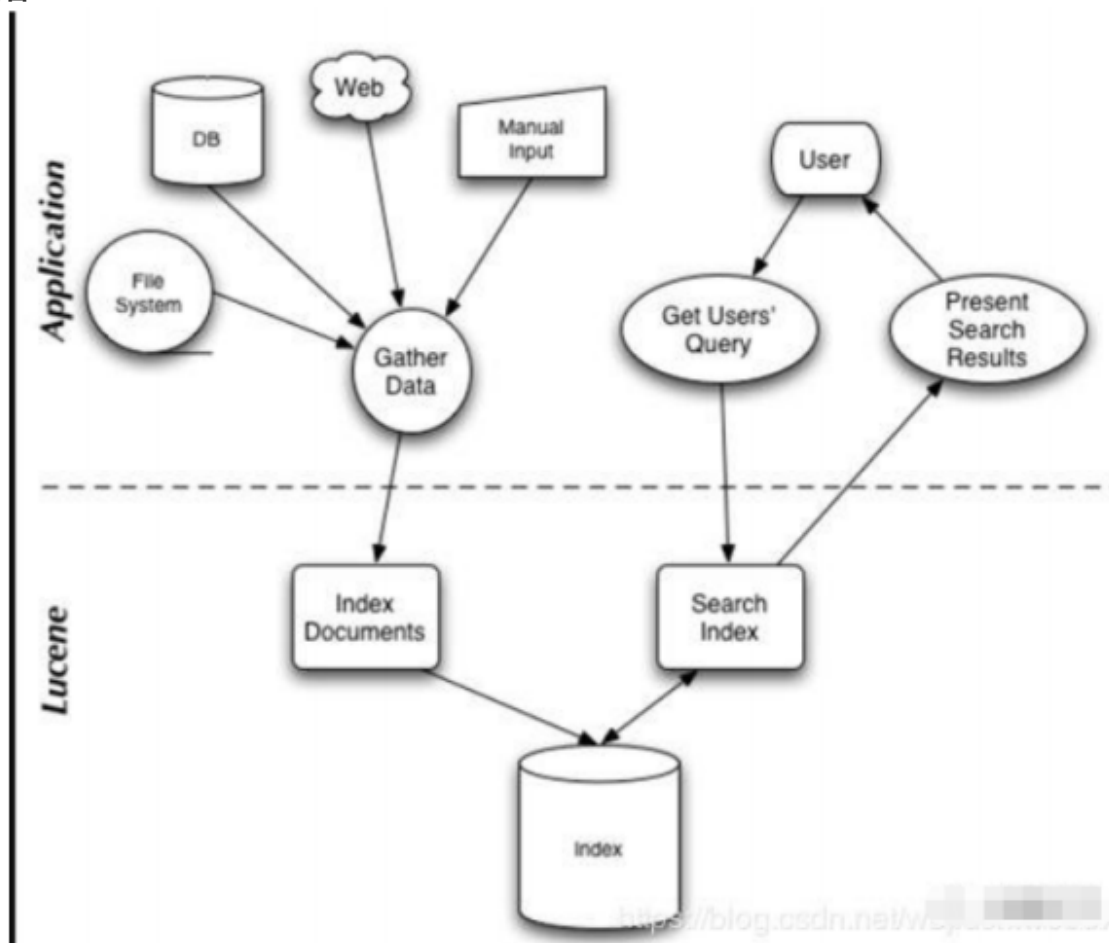
解答：

- 1、关闭缓存swap
- 2、堆内存设置为：Min（节点内存/2, 32GB）
- 3、设置最大文件句柄数
- 4、线程池+队列大小根据业务需要做调整
- 5、磁盘存储raid方式——存储有条件使用RAID10，增加单节点性能以及避免单节点存储故障

聊聊：lucence内部结构是什么？

面试官：想了解你的知识面的广度和深度。

解答：



Lucene是有索引和搜索的两个过程，包含索引创建，索引，搜索三个要点。可以基于这个脉络展开一些。

最近面试一些公司，被问到的关于Elasticsearch和搜索引擎相关的问题，以及自己总结的回答。

聊聊：Elasticsearch是如何实现Master选举的？

- 1、Elasticsearch的选主是ZenDiscovery 模块负责的，主要包含Ping（节点之间通过这个RPC来发现彼此）和Unicast（单播模块包含一个主机列表以控制哪些节点需要ping通）这两部分；
- 2、对所有可以成为master的节点（node.master: true）根据nodeId字典排序，每次选举每个节点都把自己所知道节点排一次序，然后选出第一个（第0位）节点，暂且认为它是master节点。
- 3、如果对某个节点的投票数达到一定的值（可以成为master节点数 $n/2+1$ ）并且该节点自己也选举自己，那这个节点就是 master。否则重新选举一直到满足上述条件。
- 4、补充：master节点的职责主要包括集群、节点和索引的管理，不负责文档级别的管理；data节点可以关闭http 功能*。

聊聊：Elasticsearch中的节点（比如共20个），其中的10个选了一个master，另外10个选了另一个master，怎么办？

- 1、当集群master候选数量不小于3个时，可以通过设置最少投票通过数量（`discovery.zen.minimum_master_nodes`）超过所有候选节点一半以上来解决脑裂问题；
- 2、当候选数量为两个时，只能修改为唯一的一个master候选，其他作为data节点，避免脑裂问题

聊聊：客户端在和集群连接时，如何选择特定的节点执行请求的？

答：

- 1、TransportClient利用transport模块远程连接一个elasticsearch集群。它并不加入到集群中，只是简单的获得一个或者多个初始化的transport地址，并以轮询的方式与这些地址进行通信。

聊聊：Elasticsearch在部署时，对Linux的设置有哪些优化方法？

答：

- 1、64GB内存的机器是非常理想的，但是32GB和16GB机器也是很常见的。少于8GB会适得其反。
- 2、如果你要在更快的CPUs和更多的核心之间选择，选择更多的核心更好。多个内核提供的额外[并发](#)远胜过稍微快一点点的时钟频率。
- 3、如果你负担得起SSD，它将远远超出任何旋转介质。基于SSD的节点，查询和索引性能都有提升。如果你负担得起，SSD是一个好的选择。
- 4、即使数据中心们近在咫尺，也要避免集群跨越多个数据中心。绝对要避免集群跨越大的地理距离。
- 5、请确保运行你应用程序的[JVM](#)和服务器的JVM是完全一样的。在Elasticsearch的几个地方，使用Java的本地序列化。
- 6、通过设置 `gateway.recover_after_nodes`、`gateway.expected_nodes`、`gateway.recover_after_time` 可以在集群重启的时候避免过多的分片交换，这可能会让数据恢复从数个小时缩短为几秒钟。
- 7、Elasticsearch 默认被配置为使用单播发现，以防止节点无意中加入集群。只有在同一台机器上运行的节点才会自动组成集群。最好使用单播代替组播。
- 8、不要随意修改垃圾回收器（CMS）和各个线程池的大小。
- 9、把你的内存的（少于）一半给Lucene（但不要超过32GB！），通过ES_HEAP_SIZE 环境变量设置。
- 10、内存交换到磁盘对服务器性能来说是致命的。如果内存交换到磁盘上，一个100微秒的操作可能变成10毫秒。再想想那么多10微秒的操作时延累加起来。不难看出swapping对于性能是多么可怕。
- 11、Lucene使用了大量的文件。同时，Elasticsearch在节点和HTTP客户端之间进行通信也使用了大量的套接字。所有这一切都需要足够的文件描述符。你应该增加你的文件描述符，设置一个很大的值，如64,000。

补充：索引阶段性能提升方法

- 1、使用批量请求并调整其大小：每次批量数据5-15 MB大是个不错的起始点。

2、存储：使用 SSD

3、段和合并：Elasticsearch 默认值是20MB/s，对机械磁盘应该是个不错的设置。如果你用的是SSD，可以考虑提高到 100~200 MB/s。

如果你在做批量导入，完全不在意搜索，你可以彻底关掉合并限流。另外还可以增加 index.translog.flush_threshold_size设置，从默认的512MB到更大一些的值，比如1GB，这可以在一次清空触发的时候在事务日志里积累出更大的段。

4、如果你的搜索结果不需要近实时的准确度，考虑把每个索引的index.refresh_interval 改到30s。

5、如果你在做大批量导入，考虑通过设置 index.number_of_replicas: 0 关闭副本。

聊聊：对于GC方面，在使用Elasticsearch时要注意什么？

答：

1、官方资料：<https://elasticsearch.cn/article/32>

2、倒排词典的索引需要常驻内存，无法GC，需要监控data node 上segmentmemory增长趋势。

3、各类缓存，field cache, filter cache, indexing cache, bulk queue 等等，要设置合理的大小，并且要应该根据最坏的情况来看heap是否够用，也就是各类缓存全部占满的时候，还有heap空间可以分配给其他任务吗？避免采用clear cache等“自欺欺人”的方式来释放内存。

4、避免返回大量结果集的搜索与聚合。确实需要大量拉取数据的场景，可以采用scan & scroll api来实现。

5、cluster stats驻留内存并无法水平扩展，超大规模集群可以考虑分拆成多个集群通过tribe node连接。

6、想知道heap够不够，必须结合实际应用场景，并对集群的heap使用情况做持续的监控。

聊聊：Elasticsearch对于大数据量（上亿量级）的聚合如何实现？

答：

Elasticsearch提供的首个近似聚合是cardinality度量。它提供一个字段的基数，即该字段的distinct或者unique值的数目。

它是基于HLL算法的。HLL会先对我们的输入作哈希运算，然后根据哈希运算的结果中的bits做概率估算从而得到基数。

其特点是：可配置的精度，用来控制内存的使用（更精确 = 更多内存）；

小的数据集精度是非常高的；我们可以通过配置参数，来设置去重需要的固定内存使用量。无论数千还是数十亿的唯一值，内存使用量只与你配置的精确度相关。

聊聊：在并发情况下，Elasticsearch如果保证读写一致？

答：

1、可以通过版本号使用乐观并发控制，以确保新版本不会被旧版本覆盖，由应用层来处理具体的冲突；

2、另外对于写操作，一致性级别支持quorum/one/all，默认为quorum，即只有当大多数分片可用时才允许写操作。但即使大多数可用，也可能存在因为网络等原因导致写入副本失败，这样该副本被认为故障，分片将会在一个不同的节点上重建。

3、对于读操作，可以设置replication为sync(默认)，这使得操作在主分片和副本分片都完成后才会返回；如果设置replication 为async时，也可以通过设置搜索请求参数_preference 为 primary来查询主分片，确保文档是最新版本。

聊聊：并发情况下，Elasticsearch 如果保证读写一致？

- 可以通过**版本号使用乐观锁并发控制**，以确保新版本不会被旧版本覆盖，由应用层来处理具体的冲突；
- 对于写操作：一致性级别支持 quorum/one/all，默认为 quorum。
 - quorum：即**只有当大多数（一半以上）分片可用时才允许写操作**。但即使大多数可用，也可能存在因为网络等原因导致写入副本失败，这样该副本被认为故障，分片将会在一个不同的节点上重建。
 - one：即只要主分片数据保存成功，那么客户端就可以进行查询操作了。
 - all：是最高的一致性级别，要求所有分片的数据要全部保存成功，才可以继续进行。
- 对于读操作：可以设置 replication 为 sync(默认为同步)，这使得操作在主分片和副本分片都完成后才会返回；设置 replication 为 async（异步）时，也可以通过设置搜索请求参数_preference 为 primary 来查询主分片，确保文档是最新版本。

聊聊：Elasticsearch 中的节点（比如共 20 个），其中的 10 个选了一个master，另外 10 个选了另一个 master，怎么办？

- 1、当集群master候选数量不小于3个时，可以通过设置最少投票通过数量（discovery.zen.minimum_master_nodes）超过所有候选节点一半以上来解决脑裂问题；
- 2、当候选数量为两个时，只能修改为唯一的一个master候选，其他作为data节点，避免脑裂问题。

聊聊：Elasticsearch 集群脑裂问题？有哪些解决方法？

“脑裂”问题可能的成因：（有两个master）

- 网络问题：集群间的网络延迟导致一些节点访问不到 master，**认为 master 挂掉了从而选举出新的master**，并对 master 上的分片和副本标红，分配新的主分片
- 节点负载：主节点的角色既为 master 又为 data，**访问量较大时可能会导致 ES 停止响应造成大面积延迟**，此时其他节点得不到主节点的响应认为主节点挂掉了，会重新选取主节点。
- 内存回收：data 节点上的 ES 进程占用的内存较大，**引发 JVM 的大规模内存回收，造成 ES 进程失去响应**。

脑裂问题解决方案

- 减少误判：discovery.zen.ping_timeout 节点状态的响应时间（**超过这个时间就会重新选举 master**），默认为 3s，可以适当调大，如果 master在**该响应时间的范围内没有做出响应应答**，判断该节点已经挂掉了。调大参数（如 6s，discovery.zen.ping_timeout:6），可适当减少误判。
- 选举触发：discovery.zen.minimum_master_nodes:1

该参数是用于控制选举行为发生的**最小集群主节点数量**。当备选主节点的个数大于等于该参数的值，且备选主节点中**有该参数个节点认为主节点挂了**，进行选举。官方建议为 $(n/2) + 1$ ，n 为主节点个数（即有资格成为主节点的节点个数）

- 角色分离：即 master 节点与 data 节点分离，限制角色
主节点配置为：node.master: true node.data: false
从节点配置为：node.master: false node.data: true

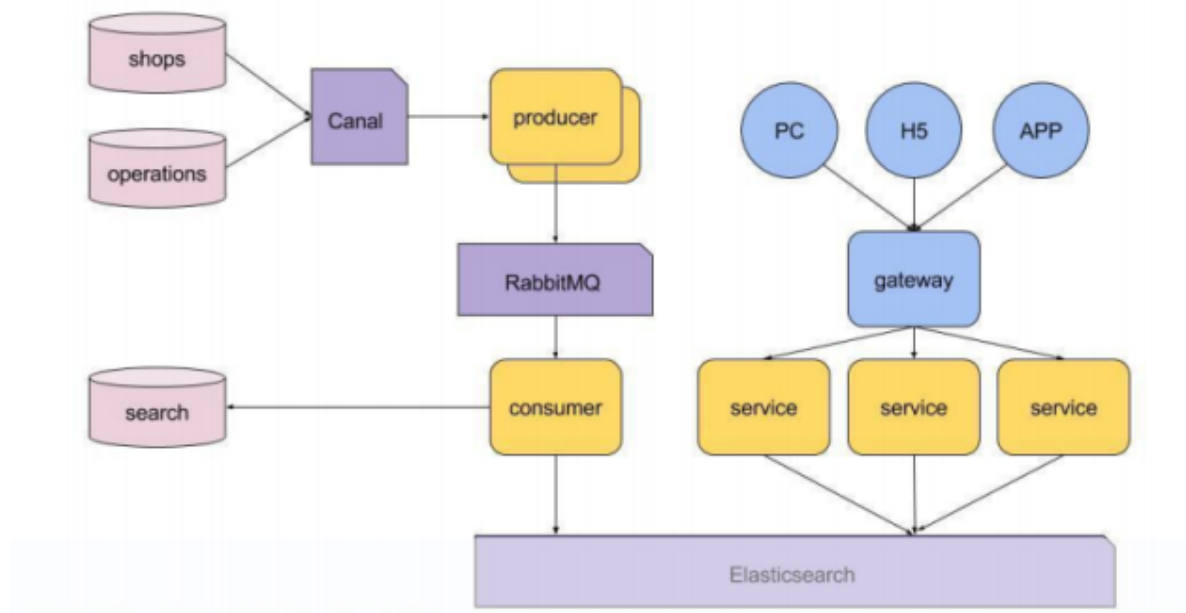
聊聊：如何监控Elasticsearch集群状态？

答：

Marvel让你可以很简单的通过Kibana监控Elasticsearch。你可以实时查看你的集群健康状态和性能，也可以分析过去的集群、索引和节点指标。

聊聊：介绍下你们电商搜索的整体技术架构。

答：



聊聊：字典树的大致思想和性质？

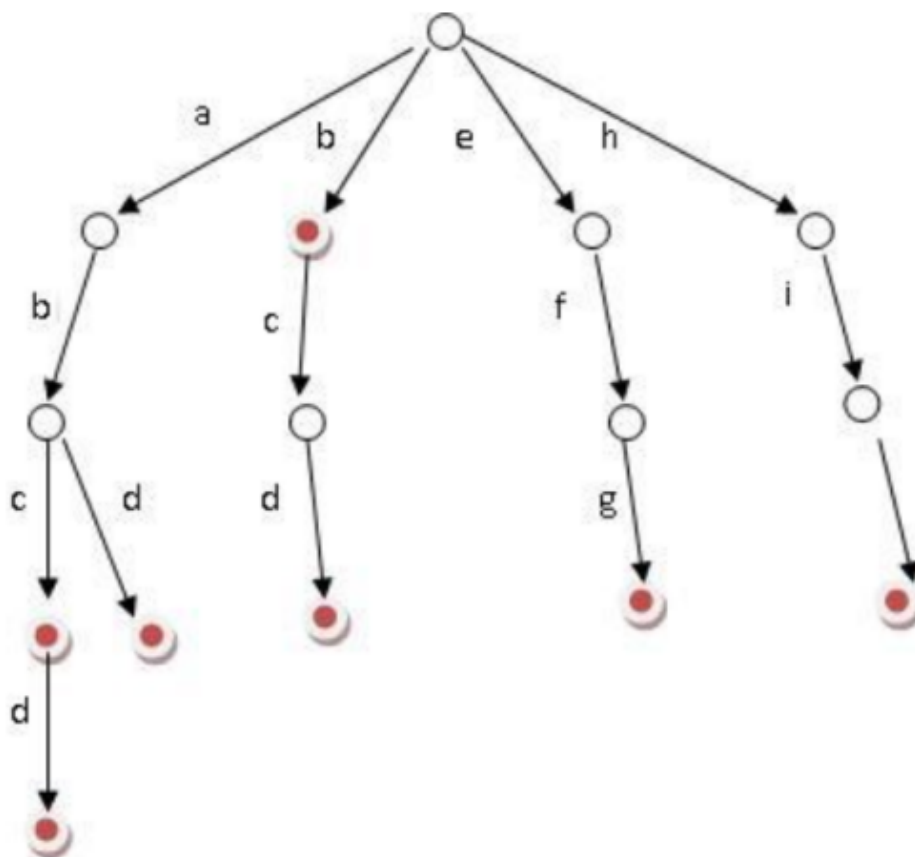
很多数据结构均能完成字典功能，总结如下。

数据结构	优缺点
排序列表Array/List	使用二分法查找，不平衡
HashMap/TreeMap	性能高，内存消耗大，几乎是原始数据的三倍
Skip List	跳跃表，可快速查找词语，在lucene、redis、Hbase等均有实现。相对于TreeMap等结构，特别适合高并发场景
Trie	适合英文词典，如果系统中存在大量字符串且这些字符串基本没有公共前缀，则相应的trie树将非常消耗内存
Double Array Trie	适合做中文词典，内存占用小，很多分词工具均采用此种算法
Ternary Search Tree	三叉树，每一个node有3个节点，兼具省空间和查询快的优点
Finite State Transducers (FST)	一种有限状态转移机，Lucene 4有开源实现，并大量使用

Trie的核心思想是空间换时间，利用字符串的公共前缀来降低查询时间的开销以达到提高效率的目的。

它有3个基本性质：

- 1、根节点不包含字符，除根节点外每一个节点都只包含一个字符。
- 2、从根节点到某一节点，路径上经过的字符连接起来，为该节点对应的字符串。
- 3、每个节点的所有子节点包含的字符都不相同。



- 1、可以看到，trie树每一层的节点数是 26^i 级别的。所以为了节省空间，我们还可以用动态链表，或者用数组来模拟动态。而空间的花费，不会超过单词数×单词长度。
- 2、实现：对每个结点开一个字母集大小的数组，每个结点挂一个链表，使用左儿子右兄弟表示法记录这棵树；
- 3、对于中文的字典树，每个节点的子节点用一个哈希表存储，这样就不用浪费太大的空间，而且查询速度上可以保留哈希的复杂度 $O(1)$

聊聊：是否了解字典树？

常用字典数据结构

- 排序列表Array/List：使用二分法查找，不平衡
- HashMap/TreeMap：性能高，内存消耗大，几乎是原始数据的三倍
- Skip List 跳跃表：可快速查找词语，在lucene、redis、Hbase等均有实现。相对于TreeMap等结构，特别
适合高并发场景(Skip List介绍)
- Trie：适合英文词典，如果系统中存在大量字符串且这些字符串基本没有公共前缀，则相应的trie树将非常消耗内存(数据结构之trie树)
- Double Array Trie：适合做中文词典，内存占用小。很多分词工具均采用此种算法(深入双数组Trie)
- Ternary Search Tree 三叉树：每一个node有3个节点，兼具省空间和查询快的优点(Ternary Search Tree)
- Finite State Transducers(FST)：一种有限状态转移机，Lucene4有开源实现，并大量使用

字典树又称单词查找树，**Trie 树**，是一种树形结构，是一种哈希树的变种。典型应用是用于统计，排序和保存大量的字符串（但不仅限于字符串），所以经常被搜索引擎系统用于文本词频统计。

它的优点是：**利用字符串的公共前缀**来减少查询时间，最大限度地减少无谓的字符串比较，**查询效率比哈希树高**（空间换时间）。

Trie 的核心思想是空间换时间，利用字符串的公共前缀来降低查询时间的开销以达到提高效率的目的。

它有 3 个基本性质：

- 根节点不包含字符，除根节点外每一个节点都只包含一个字符。
- 从根节点到某一节点，路径上经过的字符连接起来，为该节点对应的字符串。
- 每个节点的所有子节点包含的字符都不相同。

对于中文的字典树，每个节点的子节点用一个哈希表存储，这样就不用浪费太大的空间，而且查询速度上可以保留哈希的复杂度 **$O(1)$** 。

聊聊： FST原理

Finite State Transducers(FST)：一种有限状态转移机，Lucene4有开源实现，并大量使用。

lucene从4开始大量使用的数据结构是FST（Finite State Transducer）。

FST有两个优点：

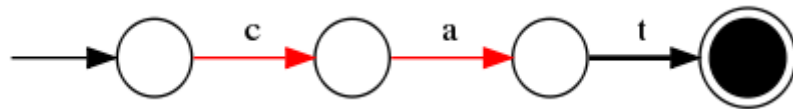
- 1) 空间占用小。通过对词典中单词前缀和后缀的重复利用，压缩了存储空间；
- 2) 查询速度快。 $O(\text{len}(\text{str}))$ 的查询时间复杂度。

下面简单描述下FST的构造过程。

我们对“cat”、“deep”、“do”、“dog”、“dogs”这5个单词进行插入构建FST（注：必须已排序）。

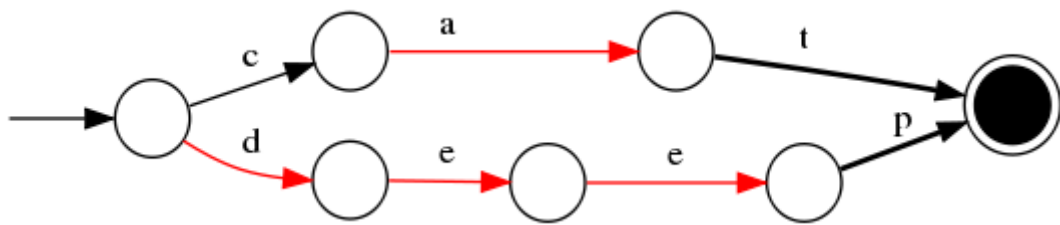
1) 插入“cat”

插入cat，每个字母形成一条边，其中t边指向终点。



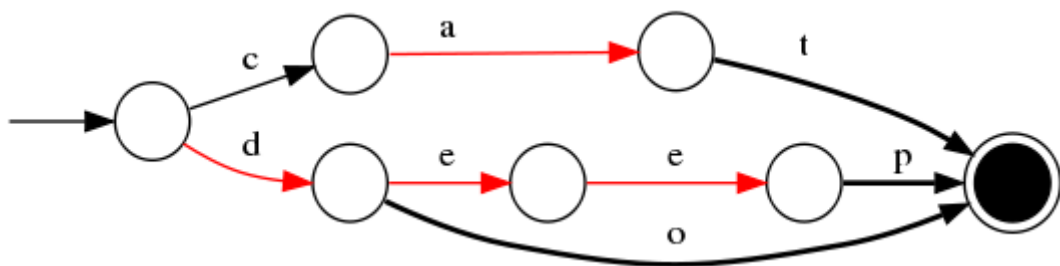
2) 插入“deep”

与前一个单词“cat”进行最大前缀匹配，发现没有匹配则直接插入，P边指向终点。



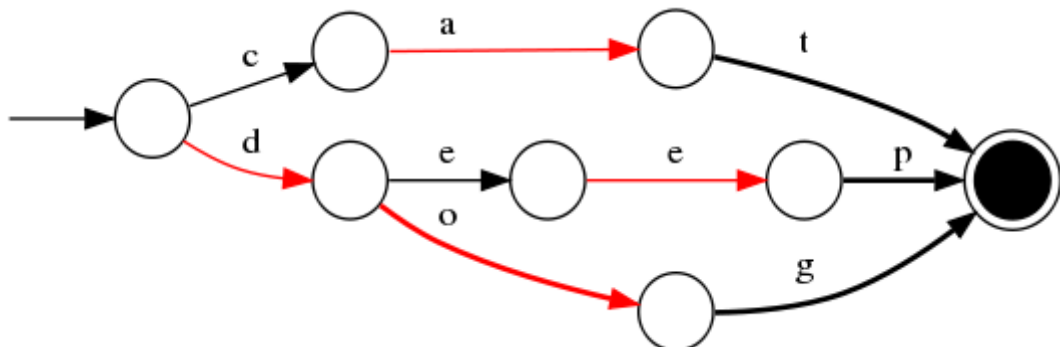
3) 插入“do”

与前一个单词“deep”进行最大前缀匹配，发现是d，则在d边后增加新边o，o边指向终点。



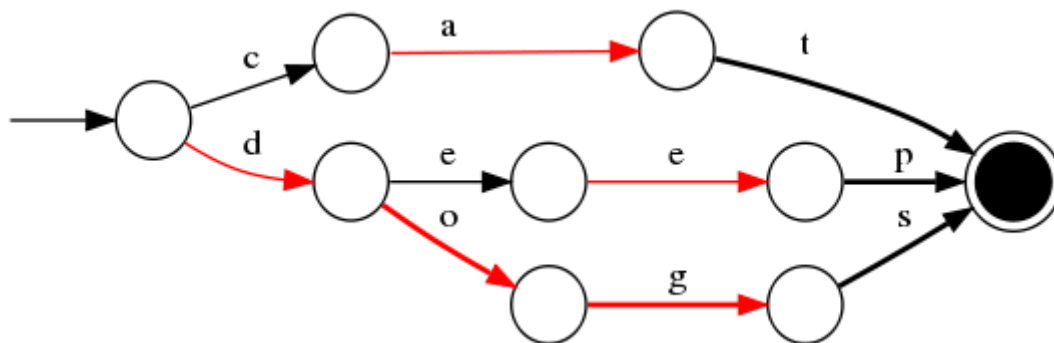
4) 插入“dog”

与前一个单词“do”进行最大前缀匹配，发现是do，则在o边后增加新边g，g边指向终点。



5) 插入“dogs”

与前一个单词“dog”进行最大前缀匹配，发现是dog，则在g后增加新边s，s边指向终点。



最终我们得到了如上一个有向无环图。利用该结构可以很方便的进行查询，如给定一个term “dog”，我们可以通过上述结构很方便的查询存不存在，甚至我们在构建过程中可以将单词与某一数字、单词进行关联，从而实现key-value的映射。

聊聊：拼写纠错是如何实现的？

答：

1、拼写纠错是基于编辑距离来实现；编辑距离是一种标准的方法，它用来表示经过插入、删除和替换操作从一个字符串转换到另外一个字符串的最小操作步数；

2、编辑距离的计算过程：比如要计算 batyu 和 beauty 的编辑距离，先创建一个7×8 的表（batyu长度为5，coffee长度为6，各加2），接着，在如下位置填入黑色数字。其他格的计算过程是取以下三个值的最小值：

如果最上方的字符等于最左方的字符，则为左上方的数字。否则为左上方的数字+1。

（对于3,3 来说为 0）

左方数字+1（对于3,3格来说为2）

上方数字+1（对于3,3格来说为2）

最终取右下角的值即为编辑距离的值3。

		b	e	a	u	t	y
	0	1	2	3	4	5	6
b	1	0	1	2	3	4	5
a	2	1	1	1	2	3	4
t	3	2	2	2	2	2	3
y	4	3	3	3	3	3	2
u	5	4	4	4	3	4	3

对于拼写纠错，我们考虑构造一个度量空间（Metric Space），该空间内任何关系满足以下三条基本条件：

$d(x,y) = 0$ - 假如 x 与 y 的距离为 0，则 $x=y$

$d(x,y) = d(y,x)$ - x 到 y 的距离等同于 y 到 x 的距离

$d(x,y) + d(y,z) \geq d(x,z)$ - 三角不等式

1、根据三角不等式，则满足与query距离在n范围内的另一个字符转B，其与A的距离最大为d+n，最小为d-n。

2、BK树的构造就过程如下：

每个节点有任意个子节点，每条边有个值表示编辑距离。所有子节点到父节点的边上标注n表示编辑距离恰好为n。

比如，我们有棵树父节点是“book”和两个子节点“cake”和“books”，“book”到“books”的边标号

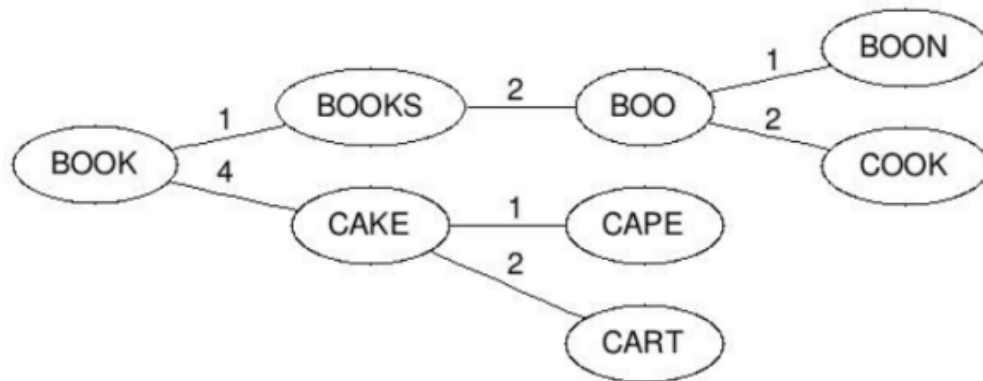
1，“book”到“cake”的边上标号4。

从字典里构造好树后，无论何时你想插入新单词时，计算该单词与根节点的编辑距离，并且查找数值为 $d(\text{newword}, \text{root})$ 的边。递归得与各子节点进行比较，直到没有子节点，你就可以创建新的子节点并将新单词保存在那。

比如，插入“boo”到刚才上述例子的树中，我们先检查根节点，查找 $d(\text{“book”}, \text{“boo”})=1$ 的边，然后检查标号为1的边的子节点，得到单词“books”。我们再计算距离 $d(\text{“books”}, \text{“boo”})=2$ ，则将新单词插在“books”之后，边标号为2。

3、查询相似词如下：计算单词与根节点的编辑距离 d ，然后递归查找每个子节点标号为 $d-n$ 到 $d+n$ （包含）的边。假如被检查的节点与搜索单词的距离 d 小于 n ，则返回该节点并继续查询。

比如输入cape且最大容忍距离为1，则先计算和根的编辑距离 $d(\text{"book"}, \text{"cape"})=4$ ，然后接着找和根节点之间编辑距离为3到5 的，这个就找到了cake这个节点，计算 $d(\text{"cake"}, \text{"cape"})=1$ ，满足条件所以返回cake，然后再找和cake节点编辑距离是0到2 的，分别找到cape和cart节点，这样就得到cape这个满足条件的结果。



参考文献：

https://www.infoq.cn/article/database-timestamp-02?utm_source=infoq&utm_medium=related_content_link&utm_campaign=relatedContent_articles_clk

<https://www.cnblogs.com/sha0830/p/8000242.html>

https://blog.csdn.net/andy_wcl/article/details/81631609

<https://cloud.tencent.com/developer/news/329497>

https://blog.csdn.net/qq_39144436/article/details/124509108

硬核推荐：尼恩Java硬核架构班

又名疯狂创客圈社群 VIP

详情：

<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java 硬核架构班

定价19999 / 早鸟 3999
即将涨价 4999

已经发布

- 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操-含简历指导》
- 《亿级用户超高并发秒杀实操-含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，2023春招面试涨薪神器
- 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023春招面试涨薪大神器
- 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、较杀式解读 rocketmq如何保障消息的可靠性？
- 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- 《架构师实操篇：redis cluster 工业级高可用实操》
- 《架构师实操篇：100W级别QPS日志平台实操》

规划中

- 《彻底穿透：skywalking 源码（代表链路跟踪）+ Java agent + bytebuddy 探针》
- 《架构师实操篇：基于netty 手写 rpc 框架-参考 dubbo、seata rpc框架》
- 《架构师实操篇：go语言学习，以及基于 go 手写 rpc 框架》
- 《架构师实操篇：千万级任务调度平台 架构与实操-基于尼恩17年的亿级搜索项目》
- 《架构师实操篇：工业级 亿级文档搜索 平台 架构与实操-基于尼恩17年的亿级搜索项目》

特色

会员制

提供技术方向指导，
职业生涯指导，少坑，少弯路

简历指导

这个很重要，
对于涨薪来说

实操性

以上项目，都是老架构师
在生产上实操过的项目

非水货

40岁老架构师，不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

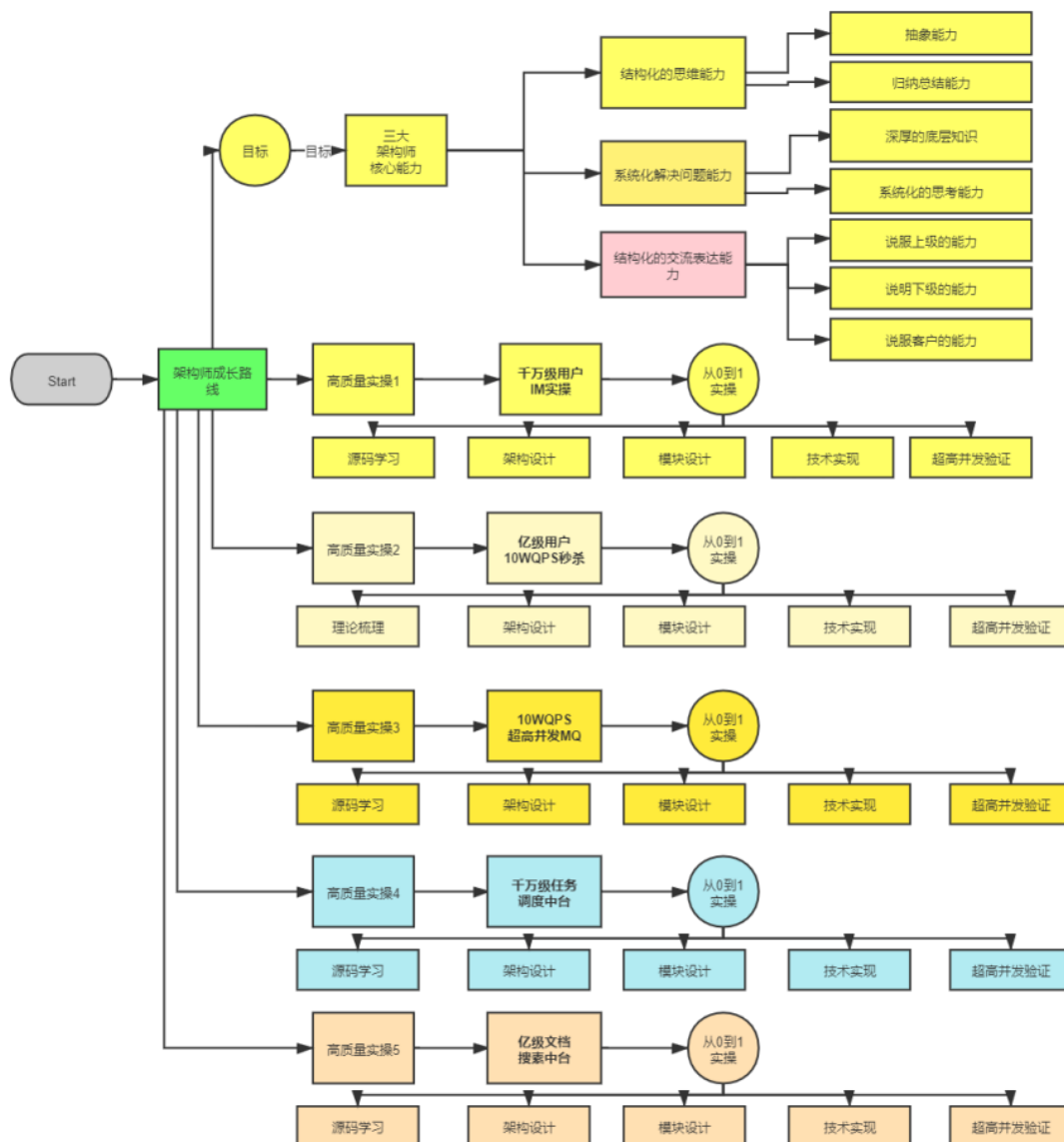
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

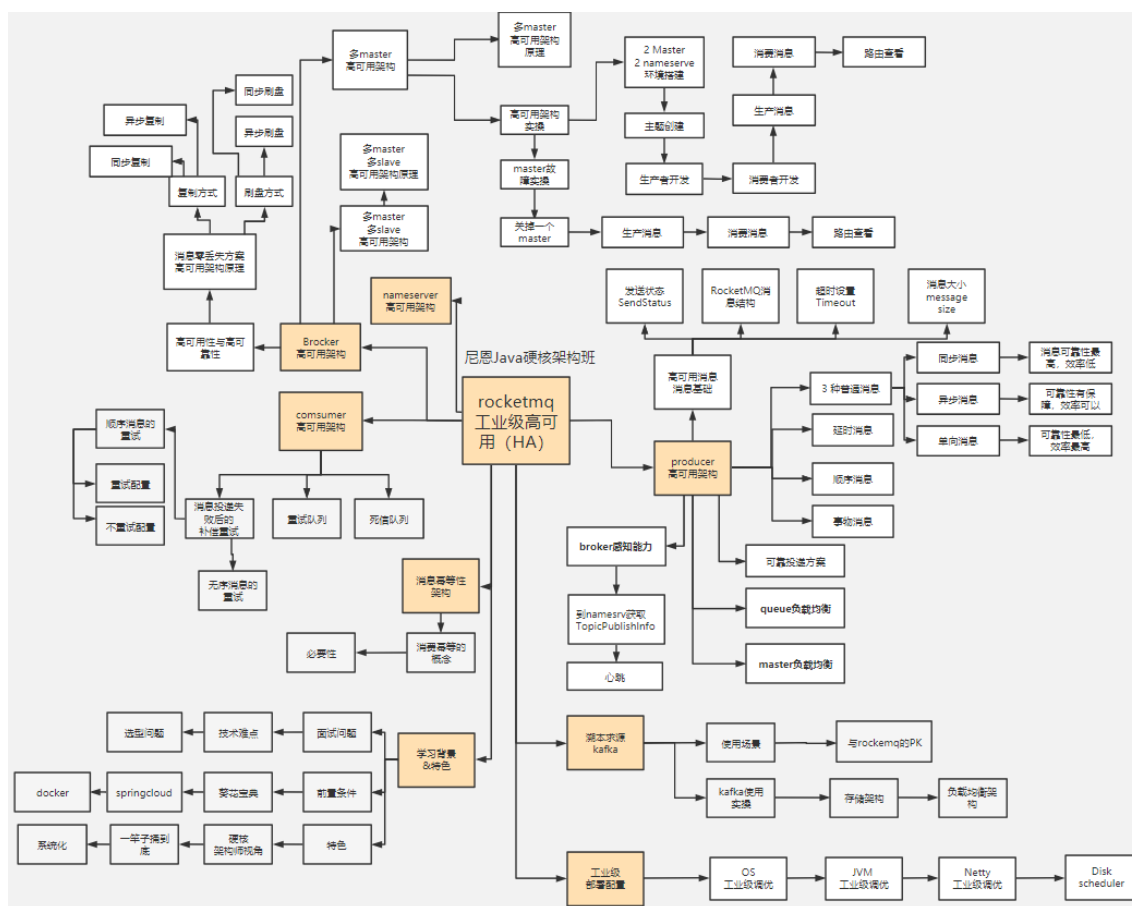
- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩





成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新	最后发表	热门	精华	最新	最后发表	热门	精华
 成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%	 卷王1号	超级版主	前天 17:41	 成功案例：[693号卷王] 二线城市6年卷王喜提4大优质Offer，含央企offer，最高薪酬35W	 卷王1号	超级版主	2022-4-16
 成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W	 卷王1号	超级版主	2022-9-21	 成功案例：[85号卷王] 双非2本小伙，春招大捷，喜提9个offer，最高薪酬近30万	 卷王1号	超级版主	2022-4-14
 成功案例：[878号卷王] 小伙8年经验，年薪60W	 卷王1号	超级版主	2022-8-13	 成功案例：[741号卷王] 卷王逆袭！6年小伙从很少面试机会到搞定35K*14薪Offer	 卷王1号	超级版主	2022-4-12
 年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W	 卷王1号	超级版主	2022-2-11	 成功案例：[642号卷王] 热烈祝贺，6年卷王喜提优质国企offer	 卷王1号	超级版主	2022-4-7
 成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬逆势涨30%	 卷王1号	超级版主	前天 17:43	 成功案例：[796号卷王] 热烈祝贺，36岁卷王喜提52万优质offer	 卷王1号	超级版主	2022-3-25
 成功案例：[250号卷王] 就业极寒时代，收offer 涨25%	 卷王1号	超级版主	前天 17:38	 成功案例：[15号卷王] 小伙卷1年，涨薪9K+，喜收ebay等多个优质offer	 卷王1号	超级版主	2022-3-24
 成功案例：[612号卷王] 就业极寒时代，从外包到白研	 卷王1号	超级版主	前天 17:15	 成功案例：[821号卷王] 小伙狠卷3个月，喜提10多个offer	 卷王1号	超级版主	2022-3-21
 成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W	 卷王1号	超级版主	2022-9-21	 成功案例：[736号卷王] 3年半经验收22k offer，但是小伙志存高远，冲击25k+	 卷王1号	超级版主	2022-3-20
 成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%	 卷王1号	超级版主	2022-9-21	 成功案例：热烈祝贺一群小卷王offer拿到手软，甚至拒了阿里offer	 卷王1号	超级版主	2022-3-16
 成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K	 卷王1号	超级版主	2022-9-21	 简历案例：简历一改，腾讯的邀请就来！热烈祝贺，小伙收到一大堆面试邀请	 卷王1号	超级版主	2022-3-10
 成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%	 卷王1号	超级版主	2022-9-21	 成功案例：祝贺我圈两大超赞卷王，一个过了阿里HR面，一个过了阿里2面	 卷王1号	超级版主	2022-3-10
 成功案例：[287号卷王] 不惧大寒潮，卷王逆市收4 offer，涨30%，可喜可贺	 卷王1号	超级版主	2022-5-30	 成功案例：小伙伴php转Java，卷1.5年Java，涨薪50%，喜收多个优质offer	 卷王1号	超级版主	2022-3-10
 成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+	 卷王1号	超级版主	2022-7-5	 成功案例：4年小伙狠卷半年，拿到 移动、京东 两大顶级offer	 卷王1号	超级版主	2022-3-5
 成功案例：[7号卷王] 热烈祝贺小伙伴涨薪120%	 卷王1号	超级版主	2022-8-13	 成功案例：[267号卷王] 助力3年经验卷王，拿到蜂巢的17k x 14薪的offer	 卷王1号	超级版主	2022-2-27
 成功案例：[134号卷王] 大三小伙卷1年，斩获顶级央企Offer，成功逆袭	 卷王1号	超级版主	2022-7-6	 成功案例：[143号卷王] 二本院校00后卷神，毕业没到一年跳到字节，年薪45W	 卷王1号	超级版主	2022-2-27
 成功案例：[1008号卷王] 5年经验卷王收42W offer，月涨8000，可喜可贺	 卷王1号	超级版主	2022-5-30	 成功案例：[494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer	 卷王1号	超级版主	2022-2-27
 成功案例：[453号卷王] 非全日制 6年卷王喜提3 offer，年薪30W，可喜可贺	 卷王1号	超级版主	2022-5-21	 成功案例：[76号卷王] 2线城市卷王，狠卷1.5年，喜收22K offer	 卷王1号	超级版主	2022-2-27
 成功案例：[924号卷王] 6年卷王喜提4 offer，最高涨薪9000，可喜可贺	 卷王1号	超级版主	2022-5-21	 成功案例：[429号卷王] 小伙伴在社群卷5个月，涨8k+	 卷王1号	超级版主	2022-2-27
 成功案例：[15号卷王] 4年卷王入职 微软，涨薪50%，可喜可贺	 卷王1号	超级版主	2022-5-12	 成功案例：[154号卷王] 双非学校毕业卷王，连拿 京东到家&滴滴 两个大厂Offer	 卷王1号	超级版主	2022-2-27
 成功案例：[527号卷王] 4年卷王喜提2 offer，涨薪50%，可喜可贺	 卷王1号	超级版主	2022-5-13	 成功案例：[232号卷王] 涨薪10K，继续卷向食物链顶端	 卷王1号	超级版主	2022-2-27
 成功案例：[788号卷王] 3年卷王喜提优质Offer，涨薪60%	 卷王1号	超级版主	2022-5-11	 成功案例：狠卷1年技术，喜收 腾讯、阿里、微软 三大Offer，最高年薪56W	 卷王1号	超级版主	2022-2-27
 成功案例：热烈祝贺：非全日制卷王，喜提2个心仪offer，面3家过2家	 卷王1号	超级版主	2022-4-21	 成功案例：[449号卷王] 应届毕业卷王喜收 滴滴offer，年薪33W	 卷王1号	超级版主	2022-2-27
 成功案例：[732号卷王] 尼恩助力3年经验卷王收获 京东offer，年薪35W	 卷王1号	超级版主	2022-2-27	 成功案例：[551号卷王] 小伙伴学完后，成功进入大厂，并且推荐自己的朋友加VIP学习	 卷王1号	超级版主	2022-2-10
 成功案例：[558号卷王] 2年经验卷王，喜收 网易和阿里子公司两个优质offer	 卷王1号	超级版主	2022-2-27	 成功案例：[214号卷王] 助力2年经验卷王，成功拿到17K月薪	 卷王1号	超级版主	2022-2-10
 成功案例：[569号卷王] 双非应届卷王，喜收字节跳动实习offer	 卷王1号	超级版主	2022-2-25	 成功案例：[92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer	 卷王1号	超级版主	2022-2-10
 成功案例：[420号卷王] 狠卷1年，卷王涨薪80%，涨薪12000元！	 卷王1号	超级版主	2022-2-25	 成功案例：社群卷王小伙伴成功过了滴滴三面 获滴滴Offer	 卷王1号	超级版主	2022-2-10
 成功案例：[76号卷王] 通过尼恩1年半的指导，专科学历小伙伴从0.8K涨到22K	 卷王1号	超级版主	2022-2-10	 [612号卷王]滴滴小伙伴，蹲点考察半年，觉得靠谱后加入 疯狂创客圈	 卷王1号	超级版主	2022-2-10

简历优化后的成功涨薪案例 (VIP 含免费简历优化)

简历优化，卷王逆袭部分成功案例

The following table summarizes the 20 successful salary increase cases shown in the grid:

Case Title	Timeline	Outcome
小伙8年经验 年薪60W	7月12日改简历, 8月10日接offer	成功涨薪
7年经验卷王 薪酬涨30%	7月11日改简历, 9月1日接offer	成功涨薪
4年经验卷王逆袭 被毕业后, 反涨24W	7月改简历, 8月30日接offer	成功涨薪
小伙5月份“被毕业”, 改简历后 新获顶级央企Offer 涨薪7000+	5月29日改简历, 7月5日接offer	成功涨薪
5年卷王喜收2大Offer 最高涨5K	5月19日改简历, 9月13日接offer	成功涨薪
6年小伙伴 年薪40W	9月6日改简历, 9月21日接offer	成功涨薪
卷王逆袭成功案例 6年小伙从很少面试机会到 搞定35K*14薪	3月9日改简历, 4月11日接offer	成功涨薪
卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W	2月9日改简历, 4月15日接offer	成功涨薪
卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k, 年薪35W	4月14日改简历, 5月17日接offer	成功涨薪
卷王逆袭成功案例 5年经验小伙收2个offer 最高涨薪8k, 年薪42W	5月9日改简历, 5月30日接offer	成功涨薪
小伙高中学历 薪酬涨120%	5月6日改简历, 7月22日接offer	成功涨薪
卷王逆袭成功案例 寒冬冻六之际卷王大逆袭 收3大offer, 涨30%	5月17日改简历, 5月27日接offer	成功涨薪
卷王逆袭成功案例 4年卷王入职微软, 涨50%	3月7日改简历, 5月12日接offer	成功涨薪
4年小伙喜收百度、Boss直聘 等N个顶级Offer 最高涨幅100%	6月27日改简历, 9月19日接offer	成功涨薪
卷王逆袭成功案例 4年卷王入收2个offer, 涨50%	3月23日改简历, 5月12日接offer	成功涨薪
卷王逆袭成功案例 非全日制卷王 面试3家 收2个offer 涨薪30%	4月13日改简历, 4月21日接offer	成功涨薪
卷王逆袭成功案例 非全日制 6年经验卷王 喜提3个Offer, 年包30W	5月9日改简历, 5月18日接offer	成功涨薪
卷王逆袭成功案例 双非二本小伙伴喜得大翻身 喜提9大offer	2月22日改简历, 4月13日接offer	成功涨薪
小伙大三暑期很焦虑 跟着尼恩卷一年 校招新获顶级央企Offer	去年5月19日加入VIP, 今年7月5日接offer	成功涨薪
卷王逆袭成功案例 3年经验卷王, 涨60%	4月16日改简历, 5月11日接offer	成功涨薪

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>